Enterprise-Grade Recommendation System with Deep Learning

Complete Project Explanation and Code Walkthrough

Domain: E-Commerce / Retail

Author	Eashwaradhinesh K
Project	Enterprise-Grade Recommendation System with Deep Learning
Domain	E-Commerce / Retail
Stack	Python, pandas, scikit-learn, PyTorch, FastAPI, Streamlit
Dataset	6,000 users • 2,500 items • 138,345 interactions (synthetic)
Best model	Hybrid 4-signal fusion — NDCG@10 = 0.1095 (+69.8% over baseline)
Code	5,071 lines across 14 modules
Document	Full explanation with line-by-line code walkthrough

How to read this document

Part 1 explains the business problem and the result. Part 2 covers architecture and data flow. **Part 3 is the code walkthrough** — every module, with the real source quoted by function name and explained block by block. Part 4 covers evaluation, Part 5 the problems encountered and how they were diagnosed, and Part 6 is a viva question bank. Every code listing in Part 3 is read directly from the repository, so it cannot drift from the actual source.

Contents

Part 1 — Project Overview

1.1 The business problem

The scenario is a digital commerce platform in an **early-stage or new-market position**. That creates a specific difficulty: real user interaction data is limited or unavailable, models must be prototyped on synthetic data, and the design has to survive future scale.

The company still needs a working recommendation system *before* live traffic exists, because recommendations drive engagement, retention and revenue. Waiting for data is not an option — the system has to be designed, prototyped and validated up front.

1.2 What the system must do

- Generate realistic synthetic e-commerce data
- Build baseline and deep learning recommenders
- Handle cold-start users and cold-start products
- Evaluate recommendation quality honestly
- Expose recommendations through an application layer

1.3 Why recommendation is hard

Recommendation is not a standard prediction task. The data has structural properties that break naive approaches, and each one had to be deliberately engineered into the synthetic dataset so the system could be tested against it:

Property	In this dataset	Why it breaks naive methods
Sparsity	99.08% of cells empty	Two users almost never rate the same item, so direct similarity has nothing to compare
Popularity bias	Top 10% of items hold 62.9% of demand	Models collapse onto best-sellers and stop being personalised
Long tail	Gini 0.716	Most inventory is nearly invisible and will never sell
Cold-start users	480 (8.0%)	No history at all, so nothing to personalise from
Cold-start items	255 (10.2%)	No sales history, so collaborative filtering cannot score them

1.4 Headline result

Model	NDCG@10	Precision@10	Recall@10	Coverage
Hybrid (4-signal)	**0.1095**	**0.0628**	**0.1372**	20.2%
Item-based CF	0.1044	0.0579	0.1241	22.2%
NCF (deep learning)	0.0663	0.0375	0.0858	1.3%
Popularity baseline	0.0645	0.0375	0.0843	1.3%
SVD	0.0462	0.0278	0.0511	9.6%
Content (TF-IDF)	0.0116	0.0073	0.0161	75.4%

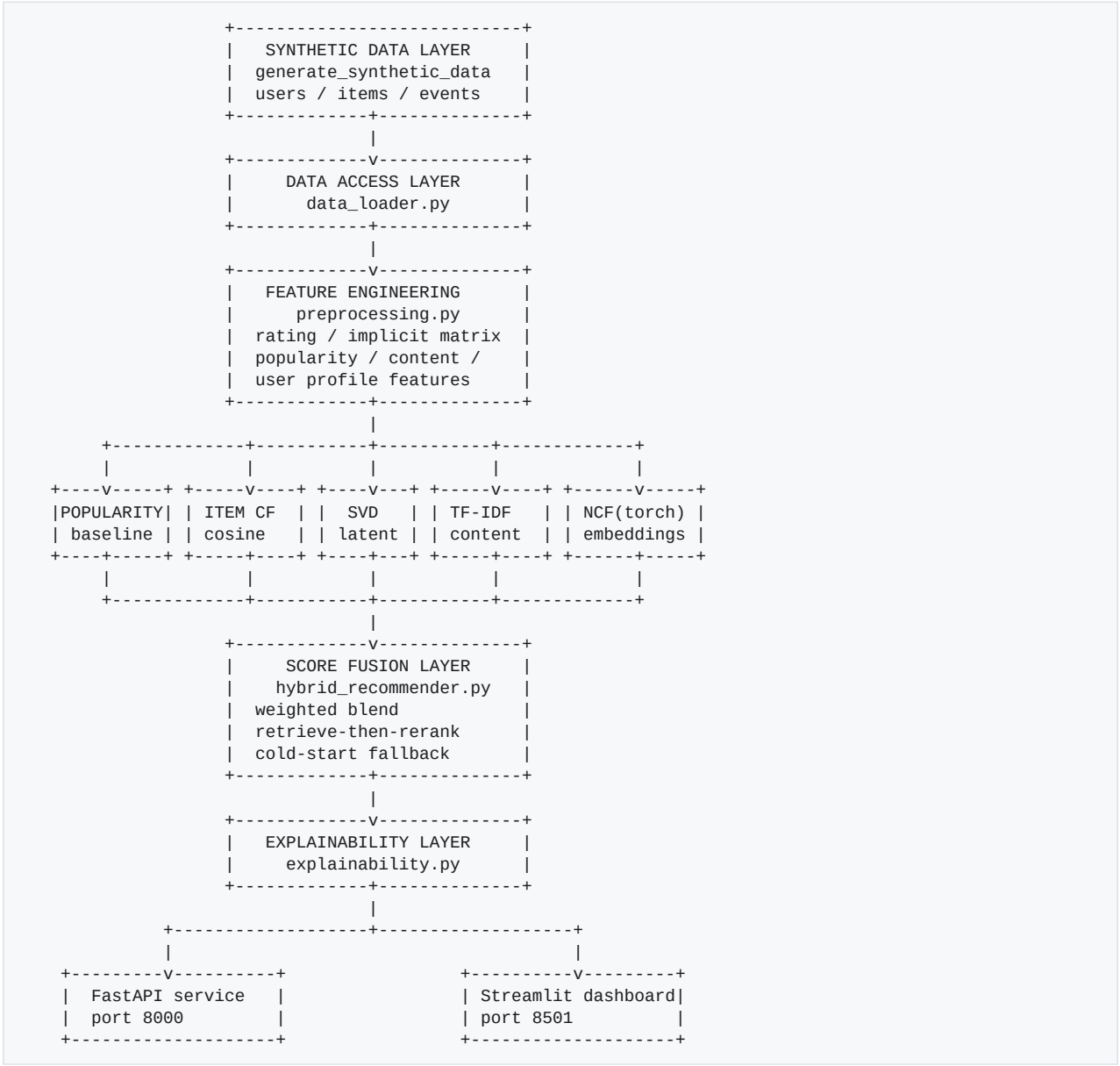
The result in one sentence

The hybrid is the best model on every ranking metric, at **NDCG@10 = 0.1095, a +69.8% lift over the popularity baseline**, measured on a time-based split with leakage assertions and all models refitted on training-period data only.

Two secondary findings are worth stating early because they shape how the rest of the document reads. First, **fusing signals genuinely helps** — the hybrid beats its own strongest component (0.1095 against 0.1044) because each signal covers the others' blind spots. Second, and more uncomfortably, **classical item-based CF beats the deep learning model on its own** (0.1044 against 0.0663) at a fraction of the cost. That is discussed honestly in Part 4.

Part 2 — Architecture and Data Flow

2.1 System architecture



All recommendation logic lives in hybrid_recommender.py. The two application layers are transport only, so they cannot return different answers.

2.2 The design rule that matters most

Every piece of recommendation logic lives in hybrid_recommender.py, packaged as a loadable service class. FastAPI and Streamlit both construct the same object. Neither contains any scoring code of its own.

This is not stylistic. The previous version of this project duplicated the fusion logic into the API file, and kept a complete copy of every source module inside the reports/ folder purely so imports would resolve. Once logic is duplicated it drifts, and two surfaces start giving different answers for the same customer. A single service class makes that impossible by construction.

2.3 Execution pipeline

#	Command	Produces	Time
1	python synthetic_data/generate_synthetic_data.py	3 raw CSVs	~10 s
2	python notebooks/eda_validation.py	7 figures, 10 checks	~30 s
3	python src/preprocessing.py	7 matrices / feature tables	~30 s
4	python src/content_based_nlp.py	TF-IDF + content similarity	~20 s
5	python models/baseline_recommenders.py	popularity, CF, SVD	~60 s
6	python src/collaborative_filtering.py	implicit CF, popularity lookup	~40 s
7	python models/ncf_recommender.py	two trained NCF models	~8 min
8	python notebooks/recommendation_evaluation.py	benchmark CSVs	~6 min

Total runtime is roughly 16 minutes on CPU. The generator is seeded, so a rebuild from an empty data/processed/ reproduces every metric in this document exactly — that was verified by wiping the folder and re-running the whole pipeline.

2.4 Repository layout

```
Enterprise-Grade-Recommendation-System-with-Deep-Learning/
|
+-- synthetic_data/
|   +-- generate_synthetic_data.py   creates users, items, interactions
+-- data/
|   +-- raw/                         users.csv, items.csv, interactions.csv
|   +-- processed/                   model artifacts (regenerated, not committed)
+-- src/
|   +-- data_loader.py               reads the three raw tables
|   +-- preprocessing.py             matrices and feature tables
|   +-- content_based_nlp.py         TF-IDF content recommender
|   +-- collaborative_filtering.py   CF improvements + re-ranking
|   +-- evaluation_metrics.py        Precision/Recall/MAP/NDCG from scratch
+-- models/
|   +-- baseline_recommenders.py     the 4 mandated baselines
|   +-- ncf_recommender.py           PyTorch Neural Collaborative Filtering
|   +-- hybrid_recommender.py        score fusion service class
|   +-- explainability.py            why an item was recommended
+-- app/
|   +-- app_fastapi.py               REST API (port 8000)
|   +-- streamlit_recommendation_dashboard.py   dashboard (port 8501)
+-- notebooks/                       5 mandated notebooks + 2 scripts
+-- reports/                         8 written reports + 7 figures
+-- requirements.txt
+-- README.md
```

2.5 How the modules depend on each other

Module	Imports from	Imported by
data_loader	— (stdlib + pandas only)	everything
preprocessing	data_loader	all models, evaluation
content_based_nlp	data_loader, preprocessing	collaborative_filtering, hybrid, explainability
collaborative_filtering	content_based_nlp, preprocessing	hybrid, evaluation

Module	Imports from	Imported by
baseline_recommenders	data_loader, preprocessing	evaluation
ncf_recommender	data_loader, preprocessing	hybrid, evaluation
hybrid_recommender	all of the above	both applications
explainability	content_based_nlp, preprocessing	both applications

The dependency graph is acyclic and shallow. data_loader depends on nothing but pandas, and every module resolves its own paths from os.path.dirname(os.path.abspath(__file__)), so the project runs from any folder or drive without configuration.

Part 3 — Code Walkthrough

Every module is walked through in execution order. Code is quoted directly from the repository with real line numbers, then explained. The focus throughout is on *why* a line is written the way it is, because that is what a reviewer asks about.

3.1 synthetic_data/generate_synthetic_data.py

751 lines — the critical phase. Everything downstream depends on this file producing data that behaves like a real platform.

3.1.1 Path handling

synthetic_data/generate_synthetic_data.py — lines 14-14

```
14 | BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

__file__ is this script's own location. os.path.dirname applied twice walks up from synthetic_data/ to the project root, so every path is derived at runtime rather than hardcoded. The previous version of this project hardcoded an absolute path to a directory that did not exist on this machine, and every module failed on import as a result.

3.1.2 Reproducibility

synthetic_data/generate_synthetic_data.py — lines 27-27

```
27 | RANDOM_SEED = 42
```

One seeded generator drives every random draw in the file, and Faker is seeded separately. np.random.default_rng(42) is the modern NumPy generator API — preferred over the legacy np.random.seed because it is a self-contained object rather than global state, so it cannot be disturbed by any library that also draws random numbers.

REFERENCE_DATE anchors the whole time window. Using a fixed date rather than datetime.now() means the dataset is identical no matter when it is generated.

3.1.3 Volume and the realism controls

synthetic_data/generate_synthetic_data.py — lines 39-39

```
39 | NUM_USERS = 6000
```

Volumes sit above the mandated minimums of 5,000 / 2,000 / 100,000 so the deliberate cold-start carve-outs still leave the dataset comfortably compliant.

synthetic_data/generate_synthetic_data.py — lines 51-51

```
51 | ZIPF_ALPHA = 1.25
```

These constants are the heart of the file. Each one is a knob that turns a flat random dataset into one with a specific real-world pathology:

Constant	Value	Controls
ZIPF_ALPHA / ZIPF_RANK_OFFSET	1.25 / 25	steepness of the popularity head
USER_ACTIVITY_LOG_MEAN / SIGMA	2.85 / 0.95	spread between power users and light users

Constant	Value	Controls
COLD_START_USER_FRACTION	0.08	share of users held at 0-2 interactions
COLD_START_ITEM_FRACTION	0.10	share of items launched recently with no sales
CATEGORY_AFFINITY_STRENGTH	0.62	how strongly a user sticks to their category
PRICE_AFFINITY_STRENGTH	0.55	how strongly a segment sticks to its price band
RATING_INTERCEPT / SLOPE	2.35 / 3.15	shape of the rating distribution
RECENCY_BIAS	1.6	concentration of activity toward recent dates

The Zipf offset — a subtle but important detail

Exposure follows 1 / (rank + 25) ** 1.25, not the textbook 1 / rank ** alpha. With a pure power law the top item's expected exposure exceeded the entire user base. Because a user can only interact with an item once, the head **saturated**: the most popular items were seen by nearly every customer and stopped carrying any discriminative signal at all. Measured before the fix, the top item drew 4,299 interactions from a 6,000-user base — 72% of all customers. The offset flattens the very top while leaving the tail untouched, giving a realistic Pareto shape without the saturation artefact.

3.1.4 Generating users

synthetic_data/generate_synthetic_data.py — lines 294-337

```
294 | def generate_users(num_users=NUM_USERS):
295 |     """Build the users table."""
296 |     segment_names = list(USER_SEGMENTS.keys())
297 |     segment_shares = np.array([USER_SEGMENTS[s]["share"] for s in segment_names])
298 |     segment_shares = segment_shares / segment_shares.sum()
299 |
300 |     segments = rng.choice(segment_names, size=num_users, p=segment_shares)
301 |
302 |     # Age correlates with segment: premium/luxury buyers skew older, because a
303 |     # flat 18-70 uniform age would carry no signal at all.
304 |     segment_age_centre = {
305 |         "Budget Shopper": 27,
306 |         "Value Seeker": 32,
307 |         "Mainstream Buyer": 37,
308 |         "Premium Buyer": 43,
309 |         "Luxury Enthusiast": 47,
310 |     }
311 |     ages = np.array([
312 |         int(np.clip(rng.normal(segment_age_centre[s], 9), 18, 72))
313 |         for s in segments
314 |     ])
315 |
316 |     genders = rng.choice(GENDERS, size=num_users, p=GENDER_WEIGHTS)
317 |     locations = rng.choice(LOCATIONS, size=num_users)
318 |     preferred_categories = rng.choice(CATEGORIES, size=num_users)
319 |
320 |     signup_offsets = rng.integers(0, HISTORY_DAYS + 365, size=num_users)
321 |     signup_dates = [
322 |         (REFERENCE_DATE - timedelta(days=int(offset))).strftime("%Y-%m-%d")
323 |         for offset in signup_offsets
324 |     ]
325 |
326 |     users_df = pd.DataFrame({
327 |         "user_id": np.arange(1, num_users + 1),
328 |         "name": [fake.name() for _ in range(num_users)],
329 |         "age": ages,
330 |         "gender": genders,
331 |         "location": locations,
332 |         "user_segment": segments,
333 |         "preferred_category": preferred_categories,
334 |         "signup_date": signup_dates,
335 |     })
336 |
337 |     return users_df
```

Segments are drawn from a weighted distribution, then **age is made conditional on segment**. A flat 18–70 uniform age would carry no information — the model could learn nothing from it. Correlating it with segment means age becomes a genuine, if weak, predictor, which is how demographics behave in reality.

signup_date spans the observation window plus a year, so tenure is a usable feature and genuinely new signups exist in the data.

3.1.5 Generating items

synthetic_data/generate_synthetic_data.py — lines 360-448

```
def generate_items(num_items=NUM_ITEMS):
    """Build the items table."""
    records = []

    # Uneven distribution across categories, since real catalogues are not
    # balanced across departments.
    category_weights = rng.dirichlet(np.ones(len(CATEGORIES)) * 6.0)
    category_assignments = rng.choice(CATEGORIES, size=num_items, p=category_weights)

    for idx, category in enumerate(category_assignments, start=1):
        spec = CATALOGUE[category]
        subcategory = str(rng.choice(spec["subcategories"]))
        brand = str(rng.choice(spec["brands"]))

        low, high = spec["price_range"]
        # Log-normal inside the band: most items cluster low, a few are premium.
        log_low, log_high = np.log(low), np.log(high)
        mu = log_low + 0.35 * (log_high - log_low)
        sigma = 0.45 * (log_high - log_low) / 3.0
        price = round(float(np.clip(np.exp(rng.normal(mu, sigma)), low, high)), 2)

        tags = list(rng.choice(spec["tags"], size=rng.integers(3, 6), replace=False))
        adjective = str(rng.choice(TITLE_ADJECTIVES))
        model_code = str(rng.integers(100, 999))

        title = brand + " " + adjective + " " + subcategory.rstrip("s") + " " + model_code

        use_case = str(rng.choice(USE_CASES))
        audience = str(rng.choice(AUDIENCES))
        feature_a, feature_b = tags[0], tags[1]

        description = (
            brand + " " + adjective + " " + subcategory + " built for " + use_case + ". "
            "This " + category + " product features " + feature_a.replace("-", " ") +
            " and " + feature_b.replace("-", " ") + " construction. "
            "Part of the " + brand + " " + subcategory + " range, it is designed for " +
            audience + " who want reliable " + subcategory.lower() + " performance. "
            "Key attributes: " + ", ".join(t.replace("-", " ") for t in tags) + "."
        )

```

```
        # Intrinsic quality: drives ratings independently of personal fit, so a
        # genuinely good product is rated well across segments.
        base_quality = round(float(np.clip(rng.beta(6, 2.5), 0.05, 0.99)), 4)

        records.append({
            "item_id": idx,
            "title": title,
            "category": category,
            "subcategory": subcategory,
            "brand": brand,
            "description": description,
            "price": price,
            "content_tags": "|".join(tags),
            "base_quality": base_quality,
        })

    items_df = pd.DataFrame(records)

    # ---- Popularity exposure (Zipf) -------------------------------------
    ranks = np.arange(1, num_items + 1)
    exposure = 1.0 / np.power(ranks + ZIPF_RANK_OFFSET, ZIPF_ALPHA)
    rng.shuffle(exposure)                       # decorrelate popularity from item_id
    items_df["exposure_weight"] = exposure

    # ---- Cold-start items -----------------------------------------------
    num_cold_items = int(num_items * COLD_START_ITEM_FRACTION)
    cold_positions = rng.choice(num_items, size=num_cold_items, replace=False)

    is_cold = np.zeros(num_items, dtype=bool)
    is_cold[cold_positions] = True
    items_df["is_cold_start_item"] = is_cold.astype(int)
    items_df.loc[is_cold, "exposure_weight"] *= 0.002

    # Cold-start items launched in the last 30 days; the rest spread out.
    launch_offsets = np.where(
        is_cold,
        rng.integers(0, 30, size=num_items),
        rng.integers(30, HISTORY_DAYS + 200, size=num_items),
    )
    items_df["launch_date"] = [
        (REFERENCE_DATE - timedelta(days=int(offset))).strftime("%Y-%m-%d")
        for offset in launch_offsets
    ]
    items_df["launch_days_ago"] = launch_offsets

    # Price percentile is used for segment price-affinity matching.
    items_df["price_percentile"] = items_df["price"].rank(pct=True)

    return items_df
```

rng.dirichlet produces an uneven split across the ten categories. Real catalogues are not balanced across departments, and a uniform split would be an obvious tell of synthetic data.

Price is drawn from a **log-normal inside each category's band**. This is what makes price a real latent signal rather than noise: Electronics has a median of INR 9,089 while Grocery sits at INR 248, and within each category most items cluster low with a few premium outliers — exactly the shape of a real catalogue.

The description field is written so that items in the same subcategory share vocabulary while items across categories do not. That is what makes TF-IDF cosine similarity meaningful later. If every description used the same filler words, every item would look equally similar to every other and the content recommender would be useless.

Three things happen here. Exposure weights are assigned by shifted Zipf and then **shuffled**, so popularity is uncorrelated with item_id — without the shuffle, low IDs would be systematically popular and any model could cheat. Cold-start items have their exposure crushed by a factor of 500 and are given launch dates inside the last 30 days, so they are coherent as newly listed inventory rather than arbitrarily starved.

3.1.6 The Gumbel top-k sampling trick

synthetic_data/generate_synthetic_data.py — lines 474-477

```
474 | def weighted_sample_without_replacement(log_weights, k):
475 |     """Gumbel top-k trick."""
476 |     keys = log_weights + rng.gumbel(size=log_weights.shape[0])
477 |     return np.argpartition(-keys, k - 1)[:k]
```

This is the performance-critical function. Adding Gumbel noise to log-weights and taking the top k is mathematically equivalent to sequential weighted sampling *without replacement*, but runs as a single vectorised operation.

> **Why this matters**
>
> The original version of this project sampled interactions with a while loop calling df.sample(1) and rejecting duplicates. That is O(n) DataFrame operations for 138,000 rows and takes several minutes. The Gumbel approach generates the entire dataset in about ten seconds.

3.1.7 The preference model

synthetic_data/generate_synthetic_data.py — lines 480-625

```
480 | def generate_interactions(users_df, items_df, target_interactions=NUM_INTERACTIONS):
481 |     """Build the interactions table as a realistic e-commerce engagement funnel."""
482 |     num_users = len(users_df)
483 |     num_items = len(items_df)
484 |
485 |     item_ids = items_df["item_id"].to_numpy()
486 |     item_categories = items_df["category"].to_numpy()
487 |     item_price_pct = items_df["price_percentile"].to_numpy()
488 |     item_quality = items_df["base_quality"].to_numpy()
489 |     item_prices = items_df["price"].to_numpy()
490 |     item_launch_days_ago = items_df["launch_days_ago"].to_numpy()
491 |
492 |     log_exposure = np.log(items_df["exposure_weight"].to_numpy())
493 |
494 |     # ---- Per-user interaction budget -----------------------------------
495 |     activity = draw_user_activity(num_users, target_interactions)
496 |
497 |     num_cold_users = int(num_users * COLD_START_USER_FRACTION)
498 |     cold_user_positions = rng.choice(num_users, size=num_cold_users, replace=False)
499 |     activity[cold_user_positions] = rng.integers(0, 3, size=num_cold_users)
500 |
501 |     user_ids = users_df["user_id"].to_numpy()
502 |     user_pref_category = users_df["preferred_category"].to_numpy()
503 |     user_segments = users_df["user_segment"].to_numpy()
504 |
505 |     segment_price_pct = np.array([USER_SEGMENTS[s]["price_percentile"] for s in user_segments])
506 |     segment_purchase_prop = np.array([USER_SEGMENTS[s]["purchase_propensity"] for s in user_segments])
507 |     segment_rating_bias = np.array([USER_SEGMENTS[s]["rating_bias"] for s in user_segments])
508 |     segment_basket = np.array([USER_SEGMENTS[s]["avg_basket_size"] for s in user_segments])
509 |
510 |     all_user_idx = []
511 |     all_item_pos = []
512 |     all_affinity = []
513 |
514 |     for u in range(num_users):
515 |         k = int(activity[u])
516 |         if k &lt;= 0:
517 |             continue
518 |         k = min(k, num_items)
519 |
```

```
        # --- Preference weighting ----------------------------------------
        # Start from raw popularity exposure, then tilt toward the categories
        # and price band this specific user cares about.
        category_match = (item_categories == user_pref_category[u]).astype(float)
        category_multiplier = 1.0 + CATEGORY_AFFINITY_STRENGTH * 8.0 * category_match

        price_distance = np.abs(item_price_pct - segment_price_pct[u])
        price_multiplier = 1.0 + PRICE_AFFINITY_STRENGTH * 4.0 * np.exp(
            -(price_distance ** 2) / 0.06
        )

        log_weights = log_exposure + np.log(category_multiplier) + np.log(price_multiplier)
        chosen = weighted_sample_without_replacement(log_weights, k)

        # --- Affinity for the chosen pairs -------------------------------
        # This single latent score drives dwell time, cart, purchase and rating,
        # so the four signals stay mutually consistent (a user does not rate 5
        # stars something they bounced off in two seconds).
        affinity = (
            0.42 * category_match[chosen]
            + 0.28 * np.exp(-(price_distance[chosen] ** 2) / 0.06)
            + 0.30 * item_quality[chosen]
        )
        affinity = np.clip(affinity + rng.normal(0, 0.11, size=k), 0.0, 1.0)

        all_user_idx.append(np.full(k, u))
        all_item_pos.append(chosen)
        all_affinity.append(affinity)

    user_idx = np.concatenate(all_user_idx)
    item_pos = np.concatenate(all_item_pos)
    affinity = np.concatenate(all_affinity)
    n = len(user_idx)

    # ---- Funnel stage 1: click / dwell time ------------------------------
    click = np.ones(n, dtype=int)
    view_time = np.clip(rng.lognormal(np.log(12 + 190 * affinity), 0.55), 3, 1800).round(1)

    # ---- Funnel stage 2: add to cart -------------------------------------
    cart_prob = 1.0 / (1.0 + np.exp(-(-2.4 + 4.6 * affinity)))
```

```
    add_to_cart = (rng.random(n) &lt; cart_prob).astype(int)

    # ---- Funnel stage 3: purchase (only reachable from cart) --------------
    purchase_prob = segment_purchase_prop[user_idx] * (0.35 + 1.25 * affinity)
    purchase = ((add_to_cart == 1) &amp;amp; (rng.random(n) &lt; purchase_prob)).astype(int)

    quantity = np.where(
        purchase == 1,
        np.maximum(1, rng.poisson(segment_basket[user_idx])),
        0,
    )
    revenue = np.round(quantity * item_prices[item_pos], 2)

    # ---- Explicit feedback: star ratings ----------------------------------
    # Buyers are far more likely to leave a rating than browsers, which is why
    # ratings are dense on purchases and sparse elsewhere - as in reality.
    rating_prob = np.where(purchase == 1, 0.88, RATING_COVERAGE)
    has_rating = rng.random(n) &lt; rating_prob

    raw_rating = (
        RATING_INTERCEPT
        + RATING_SLOPE * affinity
        + segment_rating_bias[user_idx]
        + rng.normal(0, RATING_NOISE_SD, size=n)
    )
    rating = np.where(has_rating, np.clip(np.round(raw_rating), 1, 5), np.nan)

    # ---- Timestamps -------------------------------------------------------
    # Beta(RECENCY_BIAS, 1) skews toward 1, concentrating activity in recent
    # months, which is what makes a time-based train/test split meaningful.
    recency_fraction = rng.beta(RECENCY_BIAS, 1.0, size=n)
    days_ago = (HISTORY_DAYS * (1.0 - recency_fraction)).astype(int)

    # An interaction can never predate the item's launch.
    days_ago = np.maximum(np.minimum(days_ago, item_launch_days_ago[item_pos]), 0)

    seconds_offset = rng.integers(0, 86400, size=n)
    timestamps = (
        pd.to_datetime(REFERENCE_DATE)
        - pd.to_timedelta(days_ago, unit="D")
        + pd.to_timedelta(seconds_offset, unit="s")
    )

    interactions_df = pd.DataFrame({
        "user_id": user_ids[user_idx],
        "item_id": item_ids[item_pos],
        "click": click,
        "view_time_seconds": view_time,
        "add_to_cart": add_to_cart,
        "purchase": purchase,
        "quantity": quantity,
        "revenue": revenue,
        "rating": rating,
        "timestamp": timestamps,
    })

    # Implicit feedback: the model-facing binary target. A user showed genuine
    # intent if they carted or bought.
    interactions_df["implicit_feedback"] = (
        (interactions_df["add_to_cart"] == 1) | (interactions_df["purchase"] == 1)
    ).astype(int)

    interactions_df = interactions_df.sort_values(["timestamp", "user_id"]).reset_index(drop=True)
    interactions_df["timestamp"] = interactions_df["timestamp"].dt.strftime("%Y-%m-%d %H:%M:%S")

    return interactions_df
```

For each user, raw popularity exposure is tilted by two multipliers: a category match, and a Gaussian kernel on the distance between the item's price percentile and the user segment's target percentile. Working in log-space means the multipliers combine additively, which is both numerically stable and exactly what the Gumbel trick expects.

A single **affinity** score is computed per (user, item) pair from three weighted components plus noise. This one number then drives dwell time, cart, purchase and rating.

Why one latent score rather than four independent draws

Generating the four signals independently would produce contradictory rows — a customer who bounced off a page in two seconds but rated it five stars. Driving all four from one affinity score keeps them mutually consistent, which is what lets a model learn a coherent pattern from them.

3.1.8 The engagement funnel

Every row is a click by construction. Dwell time is log-normal around a mean that scales with affinity. Cart is a logistic function of affinity. Purchase is reachable *only* from cart, and is additionally gated by the segment's purchase propensity — so a Luxury Enthusiast converts more readily than a Budget Shopper, as in reality.

Ratings are far denser on purchases (88%) than on browse-only rows (65%), because buyers review and browsers rarely do. The intercept and slope are tuned so the distribution is **J-shaped** — mean 3.91 with 67% at 4–5 stars. A symmetric distribution centred on 3 is the classic give-away of synthetic data; the first calibration produced exactly that and had to be retuned.

3.1.9 Timestamps and the launch constraint

Beta(1.6, 1) skews toward 1, concentrating activity in recent months — the last 90 days hold 30.1% of all interactions, which is what makes a time-based split meaningful rather than arbitrary. The np.minimum against launch_days_ago enforces a hard constraint: **an interaction can never predate the item's launch**. Without it the data would contain customers buying products that did not yet exist.

3.2 src/data_loader.py

70 lines — small, but with one line that prevents a serious bug.

src/data_loader.py — lines 22-28

```
22 | def check_file(path: str, name: str) -&gt; None:
23 |     """Raise an actionable error if a raw file is missing."""
24 |     if not os.path.exists(path):
25 |         raise FileNotFoundError(
26 |             name + " not found at " + path + "\n"
27 |             "Generate it first:  python synthetic_data/generate_synthetic_data.py"
28 |         )
```

Paths again derive from __file__. check_file raises a FileNotFoundError carrying the exact command needed to fix the problem, rather than letting pandas emit an opaque parser error.

src/data_loader.py — lines 43-48

```
43 | def load_interactions() -&gt; pd.DataFrame:
44 |     """Load the interaction log."""
45 |     check_file(INTERACTIONS_FILE, "interactions.csv")
46 |     # parse_dates matters: as strings the time-based split compares
47 |     # lexicographically and silently produces a wrong cutoff.
48 |     return pd.read_csv(INTERACTIONS_FILE, parse_dates=["timestamp"])
```

The one line that matters

parse_dates=["timestamp"]. Without it, pandas reads timestamps as **strings**. The time-based train/test split compares timestamps with <=, and string comparison is lexicographic — it happens to work for ISO-8601 dates, so the bug would never raise an error. It would silently produce a plausible-looking but subtly wrong split. Parsing at the boundary makes the failure impossible.

3.3 src/preprocessing.py

279 lines — turns raw events into the structures models consume.

3.3.1 The explicit rating matrix

src/preprocessing.py — lines 31-42

```
31 | def create_user_item_matrix(interactions_df: pd.DataFrame) -&gt; pd.DataFrame:
32 |     """Build the user x item explicit rating matrix."""
33 |     # Ratings start at 1, so a 0 unambiguously means "no explicit signal".
34 |     rated = interactions_df.dropna(subset=["rating"])
35 |
36 |     return rated.pivot_table(
37 |         index="user_id",
38 |         columns="item_id",
39 |         values="rating",
40 |         aggfunc="mean",
41 |         fill_value=0,
42 |     ).astype(np.float32)
```

dropna(subset=["rating"]) keeps only rated rows. Unrated cells become 0 via fill_value, which is unambiguous **because the rating scale starts at 1** — a 0 can only mean 'no explicit signal', never 'rated zero'. float32 halves the memory of a 15-million-cell matrix for no modelling loss.

3.3.2 Mean-centring

src/preprocessing.py — lines 48-62

```
48 | def create_normalized_user_item_matrix(interactions_df: pd.DataFrame) -&gt; pd.DataFrame:
49 |     """Mean-centre each user's ratings so factors capture relative preference."""
50 |     rated = interactions_df.dropna(subset=["rating"]).copy()
51 |
52 |     # transform broadcasts the group mean in one pass; .apply is far slower.
53 |     user_mean = rated.groupby("user_id")["rating"].transform("mean")
54 |     rated["normalized_rating"] = rated["rating"] - user_mean
55 |
56 |     return rated.pivot_table(
57 |         index="user_id",
58 |         columns="item_id",
59 |         values="normalized_rating",
60 |         aggfunc="mean",
61 |         fill_value=0,
62 |     ).astype(np.float32)
```

Why mean-centre?

A generous rater who scores everything 4-5 and a harsh rater who scores everything 2-3 may rank items *identically*, but to a factorisation model they look like completely different people. Subtracting each user's own mean removes that personal offset so the latent factors capture relative preference rather than rating style.

Note groupby(...).transform("mean") rather than a row-wise .apply. Transform broadcasts the group mean back to every row in one vectorised pass; the .apply equivalent used in the original version was orders of magnitude slower on a log of this size.

3.3.3 Implicit feedback

src/preprocessing.py — lines 68-83

```
68 | def create_implicit_feedback_matrix(interactions_df: pd.DataFrame) -&gt; pd.DataFrame:
69 |     """Build the binary cart/purchase intent matrix."""
70 |     interactions_copy = interactions_df.copy()
71 |
72 |     if "implicit_feedback" not in interactions_copy.columns:
73 |         interactions_copy["implicit_feedback"] = (
74 |             (interactions_copy["add_to_cart"] == 1) | (interactions_copy["purchase"] == 1)
75 |         ).astype(int)
76 |
77 |     return interactions_copy.pivot_table(
78 |         index="user_id",
79 |         columns="item_id",
80 |         values="implicit_feedback",
81 |         aggfunc="max",
82 |         fill_value=0,
83 |     ).astype(np.float32)
```

A cell is 1 when the customer carted or bought. This is a genuine behavioural signal, not a rating threshold applied after the fact, which is why it remains available for the ~31% of interactions carrying no star rating at all. aggfunc="max" means any single positive event marks the pair.

3.3.4 Item popularity features

src/preprocessing.py — lines 89-127

```
 89 | def create_item_popularity_features(interactions_df: pd.DataFrame,
 90 |                                     items_df: pd.DataFrame = None) -&gt; pd.DataFrame:
 91 |     """Per-item exposure stats used by the popularity re-ranker."""
 92 |     item_stats = interactions_df.groupby("item_id").agg(
 93 |         interaction_count=("item_id", "count"),
 94 |         average_rating=("rating", "mean"),
 95 |         purchase_count=("purchase", "sum"),
 96 |         total_revenue=("revenue", "sum"),
 97 |         avg_view_time=("view_time_seconds", "mean"),
 98 |     )
 99 |
100 |     # Reindex keeps zero-interaction items, which never appear in the log
101 |     # and are exactly the cold-start cases that must not be dropped.
102 |     if items_df is not None:
103 |         item_stats = item_stats.reindex(items_df["item_id"].values)
104 |
105 |     for column in ["interaction_count", "purchase_count", "total_revenue",
106 |                    "avg_view_time", "average_rating"]:
107 |         item_stats[column] = item_stats[column].fillna(0.0)
108 |
109 |     item_stats = item_stats.reset_index()
110 |     if "index" in item_stats.columns:
111 |         item_stats = item_stats.rename(columns={"index": "item_id"})
112 |
113 |     max_count = item_stats["interaction_count"].max()
114 |     item_stats["popularity_ratio"] = (
115 |         item_stats["interaction_count"] / max_count if max_count &gt; 0 else 0.0
116 |     )
117 |
118 |     item_stats["conversion_rate"] = np.where(
119 |         item_stats["interaction_count"] &gt; 0,
120 |         item_stats["purchase_count"] / item_stats["interaction_count"],
121 |         0.0,
122 |     )
123 |
124 |     threshold = item_stats["interaction_count"].quantile(LONG_TAIL_QUANTILE)
125 |     item_stats["is_long_tail"] = (item_stats["interaction_count"] &lt;= threshold).astype(int)
126 |
127 |     return item_stats
```

The reindex is not optional

item_stats.reindex(items_df["item_id"].values) is what keeps zero-interaction items in the table. An item with no interactions **never appears in the interaction log at all**, so a plain groupby silently drops it. Those are precisely the cold-start items the system must handle, and losing them here would make the cold-start path untestable. The table has 2,500 rows; the rating matrix has 2,264 columns. That gap is the cold-start problem made visible.

conversion_rate is the commercially meaningful quality signal. Raw interaction count only measures how widely an item was *shown*; an item with modest traffic but a high buy-through rate is genuinely good and deserves promotion, while one with heavy traffic and poor conversion is merely over-exposed.

3.3.5 Relevance labels

src/preprocessing.py — lines 213-222

```
213 | def mark_relevant_interactions(interactions_df: pd.DataFrame) -&gt; pd.DataFrame:
214 |     """Flag which interactions count as a hit for the ranking metrics."""
215 |     interactions_copy = interactions_df.copy()
216 |
217 |     # Purchases count even when unrated - dropping them understates recall.
218 |     rated_high = interactions_copy["rating"] &gt;= RELEVANCE_RATING_THRESHOLD
219 |     purchased = interactions_copy["purchase"] == 1
220 |
221 |     interactions_copy["is_relevant"] = (rated_high.fillna(False) | purchased).astype(int)
222 |     return interactions_copy
```

An interaction counts as relevant if rated at or above 4 **or** purchased. Including purchases is not optional: a bought-but-unrated item is unambiguous evidence of relevance, and excluding it would understate every model's recall by roughly the share of purchases that go unrated. Measured on this dataset, including purchases recovers substantially more positive labels.

3.4 src/content_based_nlp.py

313 lines — the answer to item cold start.

3.4.1 Assembling the text document

src/content_based_nlp.py — lines 43-62

```
43 | def build_item_text(items_df: pd.DataFrame) -&gt; pd.DataFrame:
44 |     """Assemble the text document representing each item."""
45 |     items_copy = items_df.copy()
46 |
47 |     # Un-hyphenate tags so the vectoriser matches individual words too.
48 |     tag_text = (
49 |         items_copy["content_tags"].fillna("")
50 |         .str.replace("|", " ", regex=False)
51 |         .str.replace("-", " ", regex=False)
52 |     )
53 |
54 |     items_copy["content_text"] = (
55 |         items_copy["description"].fillna("") + " "
56 |         + items_copy["category"].fillna("") + " "
57 |         + items_copy["subcategory"].fillna("") + " "
58 |         + items_copy["brand"].fillna("") + " "
59 |         + tag_text
60 |     ).str.strip()
61 |
62 |     return items_copy
```

Description alone would under-weight the structured attributes, so category, subcategory, brand and tags are appended. Tags are un-hyphenated (noise-cancelling becomes noise cancelling) so the vectoriser can match the individual words as well as the phrase.

3.4.2 TF-IDF configuration

src/content_based_nlp.py — lines 68-91

```
68 | def create_tfidf_features(items_df: pd.DataFrame, max_features: int = MAX_TFIDF_FEATURES):
69 |     """Fit the TF-IDF vectoriser over the catalogue."""
70 |     items_with_text = build_item_text(items_df)
71 |
72 |     # min_df=2 drops single-product terms (model codes); bigrams keep
73 |     # multi-word attributes like "stainless steel" as one feature.
74 |     vectorizer = TfidfVectorizer(
75 |         stop_words="english",
76 |         max_features=max_features,
77 |         ngram_range=(1, 2),
78 |         min_df=2,
79 |         sublinear_tf=True,
80 |     )
81 |
82 |     tfidf_matrix = vectorizer.fit_transform(items_with_text["content_text"])
83 |
84 |     save_pickle(vectorizer, "tfidf_vectorizer.pkl")
85 |     save_pickle(tfidf_matrix, "tfidf_matrix.pkl")
86 |
87 |     items_with_text[["item_id", "title", "category", "description", "content_text"]].to_csv(
88 |         os.path.join(PROCESSED_DATA_PATH, "item_text_metadata.csv"), index=False
89 |     )
90 |
91 |     return items_with_text, vectorizer, tfidf_matrix
```

Parameter	Value	Reason
stop_words	'english'	drops 'the', 'and' — no discriminative value
ngram_range	(1, 2)	keeps 'stainless steel' as one feature, not two weak unigrams

Parameter	Value	Reason
min_df	2	drops terms in only one product — almost always a model code
sublinear_tf	True	log-scales term frequency so repetition has diminishing returns
max_features	5000	caps the vocabulary; the matrix stays small and fast

3.4.3 The user taste profile

src/content_based_nlp.py — lines 141-165

```
141 | def build_user_content_profile(user_id: int, interactions_df: pd.DataFrame,
142 |                                items_df: pd.DataFrame, tfidf_matrix,
143 |                                min_rating: int = RELEVANCE_RATING_THRESHOLD):
144 |     """Represent a user as the centroid of the items they liked."""
145 |     user_rows = interactions_df.loc[interactions_df["user_id"] == user_id]
146 |
147 |     if user_rows.empty:
148 |         return None, []
149 |
150 |     liked_mask = (user_rows["rating"] &gt;= min_rating) | (user_rows["purchase"] == 1)
151 |     liked_item_ids = user_rows.loc[liked_mask.fillna(False), "item_id"].unique().tolist()
152 |
153 |     # Return None rather than a zero vector: the caller must fall back to
154 |     # cold-start handling instead of scoring against a meaningless profile.
155 |     if not liked_item_ids:
156 |         return None, []
157 |
158 |     position_lookup = pd.Series(np.arange(len(items_df)), index=items_df["item_id"].values)
159 |     liked_positions = [position_lookup[i] for i in liked_item_ids if i in position_lookup.index]
160 |
161 |     if not liked_positions:
162 |         return None, []
163 |
164 |     profile = np.asarray(tfidf_matrix[liked_positions].mean(axis=0))
165 |     return profile, liked_item_ids
```

A customer is represented as the **centroid of the items they liked**. 'Liked' means rated at or above threshold *or* purchased — including purchases materially widens coverage, because a large share never receive a rating.

The function returns None when there is no positive signal at all, rather than an empty or zero vector. That is deliberate: it is the caller's cue to fall back to cold-start handling rather than silently score everything against a meaningless profile.

3.4.4 Cold-start detection and handling

src/content_based_nlp.py — lines 196-208

```
196 | def get_cold_start_items(interactions_df: pd.DataFrame, items_df: pd.DataFrame,
197 |                          min_interactions: int = COLD_START_MIN_INTERACTIONS) -&gt; pd.DataFrame:
198 |     """Identify items with too little history for collaborative methods."""
199 |     # Reindex against the full catalogue: zero-interaction items never
200 |     # appear in the log and would otherwise be missed.
201 |     counts = (
202 |         interactions_df.groupby("item_id").size()
203 |         .reindex(items_df["item_id"].values, fill_value=0)
204 |         .reset_index()
205 |     )
206 |     counts.columns = ["item_id", "interaction_count"]
207 |
208 |     return counts.loc[counts["interaction_count"] &lt; min_interactions].reset_index(drop=True)
```

Again the reindex against the full catalogue — an item with zero interactions is exactly the one that must not be missed.

src/content_based_nlp.py — lines 217-242

```
217 | def recommend_for_new_user_by_profile(preferred_category: str, items_df: pd.DataFrame,
218 |                                       user_segment: str = None, n_top: int = 10) -&gt; pd.DataFrame:
219 |     """First-touch recommendations from registration data alone."""
220 |     candidates = items_df.loc[
221 |         items_df["category"].str.lower() == str(preferred_category).lower()
222 |     ].copy()
223 |
224 |     if candidates.empty:
225 |         candidates = items_df.copy()
226 |
227 |     # Price-band matching stops a Budget Shopper being shown the most
228 |     # expensive item in their category.
229 |     if user_segment in SEGMENT_PRICE_PERCENTILE and "price_percentile" in candidates.columns:
230 |         target = SEGMENT_PRICE_PERCENTILE[user_segment]
231 |         candidates["price_fit"] = 1.0 - (candidates["price_percentile"] - target).abs()
232 |         quality = candidates["base_quality"] if "base_quality" in candidates.columns else 0.5
233 |         candidates["cold_start_score"] = 0.6 * candidates["price_fit"] + 0.4 * quality
234 |     else:
235 |         candidates["cold_start_score"] = (
236 |             candidates["base_quality"] if "base_quality" in candidates.columns else 0.5
237 |         )
238 |
239 |     columns = [c for c in ["item_id", "title", "category", "subcategory", "brand",
240 |                            "price", "cold_start_score"] if c in candidates.columns]
241 |
242 |     return candidates.sort_values("cold_start_score", ascending=False)[columns].head(n_top)
```

Price-aware cold start

A new customer is scored on closeness to their *segment's price band* (price_fit) blended with intrinsic quality. This is why a Budget Shopper interested in Electronics is shown items around INR 2,000 rather than the most expensive product in the category — which is exactly what a naive popularity fallback would do.

3.4.5 Explaining similarity

src/content_based_nlp.py — lines 248-266

```
248 | def top_shared_terms(item_id_a: int, item_id_b: int, items_df: pd.DataFrame,
249 |                      vectorizer, tfidf_matrix, n_terms: int = 5) -&gt; list:
250 |     """Return the TF-IDF terms driving similarity between two items."""
251 |     position_lookup = pd.Series(np.arange(len(items_df)), index=items_df["item_id"].values)
252 |
253 |     if item_id_a not in position_lookup.index or item_id_b not in position_lookup.index:
254 |         return []
255 |
256 |     vec_a = tfidf_matrix[position_lookup[item_id_a]].toarray().flatten()
257 |     vec_b = tfidf_matrix[position_lookup[item_id_b]].toarray().flatten()
258 |
259 |     contribution = vec_a * vec_b
260 |     if not contribution.any():
261 |         return []
262 |
263 |     feature_names = np.asarray(vectorizer.get_feature_names_out())
264 |     top_positions = np.argsort(-contribution)[:n_terms]
265 |
266 |     return [str(feature_names[p]) for p in top_positions if contribution[p] &gt; 0]
```

The element-wise product of two TF-IDF vectors gives each term's contribution to the cosine similarity. Taking the top terms turns 'similarity 0.83' into something a merchandiser can audit — and if the shared terms turn out to be filler vocabulary rather than real product attributes, that is a defect in the text representation, and this function is how it gets caught.

3.5 src/collaborative_filtering.py

282 lines — fixes the ways textbook CF fails on real data.

3.5.1 The popularity penalty

src/collaborative_filtering.py — lines 56-68

```
56 | def build_popularity_lookup(item_popularity_features: pd.DataFrame,
57 |                             penalty_alpha: float = POPULARITY_PENALTY_ALPHA) -&gt; pd.DataFrame:
58 |     """Precompute the per-item re-ranking multipliers."""
59 |     lookup = item_popularity_features.copy()
60 |
61 |     # Power form, not 1/(1+log1p(count)): that spans a ~5x range, wider than
62 |     # the scores it multiplies, and collapses the ranking onto obscure items.
63 |     lookup["popularity_penalty"] = 1.0 / np.power(
64 |         lookup["interaction_count"] + 1.0, penalty_alpha
65 |     )
66 |     lookup["long_tail_boost"] = np.where(lookup["is_long_tail"] == 1, LONG_TAIL_BOOST, 1.00)
67 |
68 |     return lookup.set_index("item_id")
```

A trap that was walked into and then fixed

The obvious penalty form is 1 / (1 + log1p(count)). It was tried first and it is a trap: across this catalogue it spans a **~5x range**, which is wider than the spread of the normalised relevance scores it multiplies. The penalty then dominates the ranking outright. Measured, it drove the top-10 mean item exposure from 518 interactions down to **1**, with 100% long-tail share — the system traded away all relevance for diversity and started recommending obscurity for its own sake. The power form 1 / (count + 1) ** alpha with a small alpha keeps the correction a nudge rather than an override.

3.5.2 Two-stage ranking

src/collaborative_filtering.py — lines 106-138

```
106 | def rerank_with_popularity_balance(score_series: pd.Series, popularity_lookup: pd.DataFrame,
107 |                                    candidate_pool: int = CANDIDATE_POOL_SIZE) -&gt; pd.DataFrame:
108 |     """Retrieve by relevance, then diversify within the retrieved pool."""
109 |     scores = score_series.astype(float)
110 |
111 |     # Pool first: applied catalogue-wide, the penalty promotes irrelevant
112 |     # obscure items above genuinely relevant ones.
113 |     if candidate_pool is not None and len(scores) &gt; candidate_pool:
114 |         scores = scores.nlargest(candidate_pool)
115 |
116 |     # Normalise before multiplying: SVD scores are signed, and a negative
117 |     # score times a penalty below 1 moves UP, inverting the correction.
118 |     recommendation_df = pd.DataFrame({
119 |         "raw_score": min_max_normalize(scores),
120 |         "original_score": scores,
121 |     })
122 |
123 |     available = [c for c in RERANK_COLUMNS if c in popularity_lookup.columns]
124 |     recommendation_df = recommendation_df.join(popularity_lookup[available], how="left")
125 |
126 |     for column, default in RERANK_DEFAULTS.items():
127 |         if column in recommendation_df.columns:
128 |             recommendation_df[column] = recommendation_df[column].fillna(default)
129 |         else:
130 |             recommendation_df[column] = default
131 |
132 |     recommendation_df["adjusted_score"] = (
133 |         recommendation_df["raw_score"]
134 |         * recommendation_df["popularity_penalty"]
135 |         * recommendation_df["long_tail_boost"]
136 |     )
137 |
138 |     return recommendation_df.sort_values("adjusted_score", ascending=False)
```

Two design decisions are encoded here, both learned the hard way.

Decision 1 — retrieve, then re-rank

The diversity correction applies only to the top candidate_pool (200) items by raw relevance, not across the whole catalogue. Applied catalogue-wide, a near-zero-exposure item the customer has no affinity for outranks a genuinely relevant one purely for being obscure. Restricting the pool means every item in the final list was **relevant first and diverse second**, which is the correct precedence and is the standard retrieve-then-rank split used in production systems.

Decision 2 — normalise before multiplying

A sign bug that inverted the correction

SVD runs on the mean-centred matrix, so its predictions are **signed**. Multiplying a negative score by a penalty in (0, 1) moves it *up* toward zero — silently inverting the correction for every negatively scored item. The items that should have been pushed down were being pushed up. Min-max scaling to [0, 1] first guarantees the multiplication means what it says.

3.5.3 Calibrating alpha

Alpha is the accuracy-versus-diversity dial. Rather than picking a value on intuition, it was measured — top-10 overlap against the uncorrected ranking, inside the two-stage ranker:

alpha	Overlap with uncorrected	Interpretation
0.00	100%	no correction at all
0.10	97%	barely perceptible
0.20	**92%**	**visible tail promotion, relevance intact — selected**
0.35	86%	noticeable relevance cost

3.6 models/baseline_recommenders.py

283 lines — the four mandated baselines. These set the bar.

3.6.1 Cross-module imports

models/baseline_recommenders.py — lines 11-11

```
11 | BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

Modules in models/ need the shared code in src/. Two lines of plain os and sys put it on the path. The previous version solved this by keeping a duplicate copy of every source file in a second folder — which is how two copies drift apart.

3.6.2 Baseline 1: popularity

models/baseline_recommenders.py — lines 34-67

```
34 | def build_popularity_model(interactions_df: pd.DataFrame) -&gt; pd.DataFrame:
35 |     """Rank the catalogue by a blended popularity score."""
36 |     popularity_df = interactions_df.groupby("item_id").agg(
37 |         interaction_count=("item_id", "count"),
38 |         avg_rating=("rating", "mean"),
39 |         purchase_count=("purchase", "sum"),
40 |         total_revenue=("revenue", "sum"),
41 |     ).reset_index()
42 |
43 |     popularity_df["avg_rating"] = popularity_df["avg_rating"].fillna(0.0)
44 |     popularity_df["conversion_rate"] = (
45 |         popularity_df["purchase_count"] / popularity_df["interaction_count"]
46 |     )
47 |
48 |     def scale(series: pd.Series) -&gt; pd.Series:
49 |         lo, hi = series.min(), series.max()
50 |         if hi &gt; lo:
51 |             return (series - lo) / (hi - lo)
52 |         return pd.Series(0.0, index=series.index)
53 |
54 |     # Scale each component so the weights mean what they say; unscaled,
55 |     # interaction count would swamp average rating regardless of weight.
56 |     popularity_df["popularity_score"] = (
57 |         0.50 * scale(popularity_df["interaction_count"])
58 |         + 0.30 * scale(popularity_df["avg_rating"])
59 |         + 0.20 * scale(popularity_df["conversion_rate"])
60 |     )
61 |
62 |     popularity_df = popularity_df.sort_values(
63 |         "popularity_score", ascending=False
64 |     ).reset_index(drop=True)
65 |
66 |     save_pickle(popularity_df, "popularity_model.pkl")
67 |     return popularity_df
```

Raw interaction count alone rewards items that are merely widely *exposed*. Blending in average rating and conversion rate means an item has to be both seen and actually liked to reach the top. Each component is min-max scaled first so the 0.50 / 0.30 / 0.20 weights mean what they say — without scaling, interaction count (range 0–1,377) would swamp average rating (range 1–5) regardless of the weights.

3.6.3 Baselines 2 and 3: collaborative filtering

models/baseline_recommenders.py — lines 100-127

```
100 | def recommend_user_based(user_id: int, user_item_matrix: pd.DataFrame,
101 |                          user_similarity_df: pd.DataFrame, n_top: int = 10,
102 |                          n_neighbours: int = DEFAULT_NEIGHBOURS) -&gt; list:
103 |     """Score items by what the most similar users rated highly."""
104 |     if user_id not in user_item_matrix.index:
105 |         raise ValueError("user_id " + str(user_id) + " not present in the rating matrix.")
106 |
107 |     # Top-N neighbours only: all users would drown the signal in near-zero
108 |     # similarities and collapse the output toward popularity.
109 |     similar_users = (
110 |         user_similarity_df.loc[user_id]
111 |         .drop(user_id, errors="ignore")
112 |         .sort_values(ascending=False)
113 |         .head(n_neighbours)
114 |     )
115 |
116 |     similarity_sum = similar_users.sum()
117 |     weighted_scores = user_item_matrix.loc[similar_users.index].T.dot(similar_users)
118 |
119 |     if similarity_sum &gt; 0:
120 |         weighted_scores = weighted_scores / similarity_sum
121 |
122 |     already_seen = user_item_matrix.loc[user_id]
123 |     weighted_scores = weighted_scores.drop(
124 |         already_seen[already_seen &gt; 0].index, errors="ignore"
125 |     )
126 |
127 |     return weighted_scores.sort_values(ascending=False).head(n_top).index.tolist()
```

Only the top n_neighbours (50) most similar users contribute. Using all 5,766 would drown the signal in near-zero similarities and quietly collapse the output toward a popularity ranking — a subtle failure, because the model still returns plausible items.

models/baseline_recommenders.py — lines 147-163

```
147 | def recommend_item_based(user_id: int, user_item_matrix: pd.DataFrame,
148 |                          item_similarity_df: pd.DataFrame, n_top: int = 10) -&gt; list:
149 |     """Score candidates by similarity to the items this user rated."""
150 |     if user_id not in user_item_matrix.index:
151 |         raise ValueError("user_id " + str(user_id) + " not present in the rating matrix.")
152 |
153 |     user_ratings = user_item_matrix.loc[user_id]
154 |     rated_items = user_ratings[user_ratings &gt; 0]
155 |
156 |     if rated_items.empty:
157 |         return []
158 |
159 |     scores = item_similarity_df[rated_items.index].to_numpy() @ rated_items.to_numpy()
160 |     scores = pd.Series(scores, index=item_similarity_df.index)
161 |
162 |     scores = scores.drop(rated_items.index, errors="ignore")
163 |     return scores.sort_values(ascending=False).head(n_top).index.tolist()
```

Item-based CF scores candidates by similarity to what the customer already rated, weighted by those ratings. The implementation is a single matrix-vector product rather than a Python loop over rated items.

The scalability warning

The user-user similarity matrix is **O(users²)**. At 5,766 users it is 133 MB in float32 and manageable; at 100,000 users it would be 40 GB and simply cannot be held in memory. The item-item matrix is O(items²) and grows far more slowly, because catalogues change much more slowly than user bases. **User-user CF is the first thing that breaks as the platform grows**, which is why the architecture document recommends replacing it with approximate nearest neighbours beyond roughly 50,000 users.

3.6.4 Baseline 4: matrix factorisation

models/baseline_recommenders.py — lines 169-186

```
169 | def create_svd_model(user_item_matrix: pd.DataFrame, n_components: int = SVD_COMPONENTS):
170 |     """Factorise the rating matrix into latent user and item factors."""
171 |     svd = TruncatedSVD(n_components=n_components, random_state=42)
172 |
173 |     user_factors = svd.fit_transform(user_item_matrix)
174 |     item_factors = svd.components_.T
175 |
176 |     user_factors_df = pd.DataFrame(user_factors, index=user_item_matrix.index)
177 |     item_factors_df = pd.DataFrame(item_factors, index=user_item_matrix.columns)
178 |
179 |     save_pickle(svd, "svd_model.pkl")
180 |     save_pickle(user_factors_df, "user_factors.pkl")
181 |     save_pickle(item_factors_df, "item_factors.pkl")
182 |
183 |     explained = float(svd.explained_variance_ratio_.sum())
184 |     print("  SVD explained variance ({} components): {:.2%}".format(n_components, explained))
185 |
186 |     return svd, user_factors_df, item_factors_df
```

The rating matrix is over 99% empty, so raw co-occurrence is far too thin. Compressing to 50 latent dimensions lets the model generalise across customers who liked related-but-not-identical products. Note that SVD is fitted on the **mean-centred** matrix so the factors capture relative preference.

Explained variance comes out at 33.09%. That is not a failure — on a 99% sparse matrix most of the remaining variance is noise in the unobserved cells, and pushing the component count higher fits that noise rather than signal.

3.7 models/ncf_recommender.py

609 lines — the PyTorch deep learning model, and the single most important modelling lesson in the project.

3.7.1 Why a neural model at all

Matrix factorisation scores an item as the dot product of a user vector and an item vector. That is a fixed, linear interaction: the model learns *how much* a customer likes each latent dimension, but the dimensions can only combine additively.

This dataset was generated with a genuinely non-linear rule — a customer buys when the category matches **and** the price sits in their segment's band. A dot product cannot represent a conjunction. An MLP over concatenated embeddings can. That is the claim; the benchmark tests it rather than assuming it.

3.7.2 ID encoding

models/ncf_recommender.py — lines 65-87

```
65 | def encode_ids(interactions_df: pd.DataFrame, tag: str = "ncf"):
66 |     """Map sparse IDs onto contiguous embedding indices and persist the maps."""
67 |     encoded = interactions_df.copy()
68 |
69 |     unique_users = sorted(encoded["user_id"].unique())
70 |     unique_items = sorted(encoded["item_id"].unique())
71 |
72 |     user_to_index = {uid: idx for idx, uid in enumerate(unique_users)}
73 |     item_to_index = {iid: idx for idx, iid in enumerate(unique_items)}
74 |     index_to_user = {idx: uid for uid, idx in user_to_index.items()}
75 |     index_to_item = {idx: iid for iid, idx in item_to_index.items()}
76 |
77 |     encoded["user_idx"] = encoded["user_id"].map(user_to_index)
78 |     encoded["item_idx"] = encoded["item_id"].map(item_to_index)
79 |
80 |     # Maps must persist: serving has to reproduce the same index or the
81 |     # embedding row returned belongs to someone else.
82 |     save_pickle(user_to_index, tag + "_user_to_index.pkl")
83 |     save_pickle(item_to_index, tag + "_item_to_index.pkl")
84 |     save_pickle(index_to_user, tag + "_index_to_user.pkl")
85 |     save_pickle(index_to_item, tag + "_index_to_item.pkl")
86 |
87 |     return encoded, user_to_index, item_to_index, index_to_user, index_to_item
```

nn.Embedding is a lookup table indexed 0..N-1, so raw IDs cannot be used directly. The mappings are **persisted to disk**, and that matters: serving has to reproduce exactly the same index for a given customer, or the embedding row returned is somebody else's.

3.7.3 The time-based split

models/ncf_recommender.py — lines 93-104

```
 93 | def time_based_split(interactions_df: pd.DataFrame, test_days: int = TEST_PERIOD_DAYS):
 94 |     """Hold out the most recent test_days as the test period."""
 95 |     # A random split leaks the future and inflates every metric.
 96 |     interactions_copy = interactions_df.copy()
 97 |     interactions_copy["timestamp"] = pd.to_datetime(interactions_copy["timestamp"])
 98 |
 99 |     cutoff = interactions_copy["timestamp"].max() - pd.Timedelta(days=test_days)
100 |
101 |     train_df = interactions_copy.loc[interactions_copy["timestamp"] &lt;= cutoff].copy()
102 |     test_df = interactions_copy.loc[interactions_copy["timestamp"] &gt; cutoff].copy()
103 |
104 |     return train_df, test_df, cutoff
```

Why not a random split

A random split leaks the future into training — the model sees what a customer did in November while being asked to predict their October behaviour. Every reported metric is inflated. The business case mandates a time-based split precisely to close that hole.

3.7.4 Negative sampling — the critical detail

models/ncf_recommender.py — lines 118-158

```
118 | def build_implicit_training_frame(encoded_df: pd.DataFrame, num_items: int,
119 |                                   num_negatives: int = NUM_NEGATIVES,
120 |                                   seed: int = RANDOM_SEED) -&gt; pd.DataFrame:
121 |     """Build the implicit training set with sampled negatives."""
122 |     # Without negatives the model is asked "given they clicked, did they buy?"
123 |     # - a conversion problem. Retrieval needs contrast against the catalogue.
124 |     # Measured: 53% accuracy / NDCG 0.0000 without, 86% / 0.0576 with.
125 |     rng = np.random.default_rng(seed)
126 |
127 |     positives = encoded_df.loc[encoded_df["implicit_feedback"] == 1, ["user_idx", "item_idx"]]
128 |
129 |     # Exclude everything touched, including viewed-not-carted: those are
130 |     # ambiguous, not confirmed negatives.
131 |     seen_by_user = encoded_df.groupby("user_idx")["item_idx"].apply(set)
132 |
133 |     negative_users = []
134 |     negative_items = []
135 |
136 |     for user_idx, group in positives.groupby("user_idx"):
137 |         required = len(group) * num_negatives
138 |         seen = seen_by_user.get(user_idx, set())
139 |
140 |         candidates = rng.integers(0, num_items, size=int(required * 1.5) + 16)
141 |         sampled = [int(c) for c in candidates if c not in seen][:required]
142 |
143 |         negative_users.extend([user_idx] * len(sampled))
144 |         negative_items.extend(sampled)
145 |
146 |     positive_frame = pd.DataFrame({
147 |         "user_idx": positives["user_idx"].to_numpy(),
148 |         "item_idx": positives["item_idx"].to_numpy(),
149 |         "label": 1.0,
150 |     })
151 |     negative_frame = pd.DataFrame({
152 |         "user_idx": negative_users,
153 |         "item_idx": negative_items,
154 |         "label": 0.0,
155 |     })
156 |
157 |     combined = pd.concat([positive_frame, negative_frame], ignore_index=True)
158 |     return combined.sample(frac=1.0, random_state=seed).reset_index(drop=True)
```

This is the single most important function in the deep-learning component, and getting it wrong is subtle because **the model still trains** — it just learns nothing useful.

The naive framing takes the interaction log and labels each row by whether it converted. But every row in the log is an item the platform *already chose to show* the customer. So the model is being asked *'given they clicked it, did they buy it?'* — a conversion problem. The retrieval problem a recommender must actually solve is *'out of the entire catalogue, which items would this customer engage with at all?'*, and that question has **no negative examples anywhere in the log**.

Negatives are sampled from items the customer has never touched, at 4 per positive — the ratio from the original NCF paper. Note that items the customer viewed but did not cart are **excluded** from the negative pool: those are ambiguous, not confirmed negatives, and labelling them 0 would teach the model that browsing implies disinterest.

Framing	Accuracy	F1	NDCG@10	Coverage
Observed rows only (naive)	0.534	0.350	**0.0000**	0.006
With sampled negatives	**0.858**	**0.582**	**0.0576**	0.013

Without negatives the model was barely above chance and returned the same ~14 items to every single customer.

3.7.5 The model architecture

models/ncf_recommender.py — lines 195-216

```
195 |     def __init__(self, num_users: int, num_items: int, embedding_dim: int = EMBEDDING_DIM,
196 |                  hidden_dims: list = None, dropout: float = DROPOUT):
197 |         super().__init__()
198 |
199 |         if hidden_dims is None:
200 |             hidden_dims = HIDDEN_DIMS
201 |
202 |         self.user_embedding = nn.Embedding(num_users, embedding_dim)
203 |         self.item_embedding = nn.Embedding(num_items, embedding_dim)
204 |
205 |         layers = []
206 |         input_dim = embedding_dim * 2
207 |         for hidden_dim in hidden_dims:
208 |             layers.append(nn.Linear(input_dim, hidden_dim))
209 |             layers.append(nn.ReLU())
210 |             layers.append(nn.Dropout(dropout))
211 |             input_dim = hidden_dim
212 |
213 |         self.mlp = nn.Sequential(*layers)
214 |         self.output_layer = nn.Linear(input_dim, 1)
215 |
216 |         self.initialize_weights()
```

```
user_id --&gt; Embedding(num_users, 32) --+
                                          |
                                          +--&gt; concat (64d)
                                          |          |
item_id --&gt; Embedding(num_items, 32) --+          v
                                              Linear(64 -&gt; 128) + ReLU + Dropout
                                                     |
                                              Linear(128 -&gt; 64) + ReLU + Dropout
                                                     |
                                              Linear(64 -&gt; 32)  + ReLU + Dropout
                                                     |
                                              Linear(32 -&gt; 1)  --&gt; score
```

Concatenation, not element-wise product

This is the whole point of NCF. An element-wise product hard-codes a multiplicative interaction and reduces the model back to matrix factorisation. Concatenation leaves the interaction function itself to be *learned* by the MLP.

models/ncf_recommender.py — lines 218-230

```
218 |     def initialize_weights(self) -&gt; None:
219 |         """Small embedding init, Xavier for the MLP."""
220 |         # Large random embeddings make the first epochs pure noise.
221 |         nn.init.normal_(self.user_embedding.weight, std=0.01)
222 |         nn.init.normal_(self.item_embedding.weight, std=0.01)
223 |
224 |         for module in self.mlp:
225 |             if isinstance(module, nn.Linear):
226 |                 nn.init.xavier_uniform_(module.weight)
227 |                 nn.init.zeros_(module.bias)
228 |
229 |         nn.init.xavier_uniform_(self.output_layer.weight)
230 |         nn.init.zeros_(self.output_layer.bias)
```

Embeddings are initialised with a small standard deviation (0.01) so early predictions sit near the data mean; large random embeddings make the first epochs pure noise. The MLP uses Xavier initialisation, which keeps activation variance stable across layers.

3.7.6 Early stopping

models/ncf_recommender.py — lines 247-270

```
247 | class EarlyStopping:
248 |     """Stop when validation loss stops improving, restoring the best weights."""
249 |
250 |     def __init__(self, patience: int = PATIENCE, min_delta: float = 1e-4):
251 |         self.patience = patience
252 |         self.min_delta = min_delta
253 |         self.best_loss = float("inf")
254 |         self.counter = 0
255 |         self.best_state = None
256 |         self.best_epoch = 0
257 |
258 |     def step(self, valid_loss: float, model: nn.Module, epoch: int) -&gt; bool:
259 |         """Record the epoch and report whether training should stop."""
260 |         # Restoring matters as much as stopping: otherwise you keep the
261 |         # weights from the last, worse epoch.
262 |         if valid_loss &lt; self.best_loss - self.min_delta:
263 |             self.best_loss = valid_loss
264 |             self.counter = 0
265 |             self.best_state = copy.deepcopy(model.state_dict())
266 |             self.best_epoch = epoch
267 |             return False
268 |
269 |         self.counter += 1
270 |         return self.counter &gt;= self.patience
```

Restoring the best weights matters as much as stopping. Without best_state, you stop at the right epoch but keep the weights from the last (worse) one, which defeats the purpose of having watched at all.

3.7.7 The training loop

models/ncf_recommender.py — lines 276-343

```
276 | def train_ncf_model(model: nn.Module, train_loader: DataLoader, valid_loader: DataLoader,
277 |                     feedback_type: str = "explicit", learning_rate: float = LEARNING_RATE,
278 |                     weight_decay: float = WEIGHT_DECAY, epochs: int = EPOCHS,
279 |                     patience: int = PATIENCE, device: str = None, verbose: bool = True):
280 |     """Train with Adam, weight decay and early stopping."""
281 |     if device is None:
282 |         device = get_device()
283 |
284 |     model = model.to(device)
285 |
286 |     if feedback_type == "explicit":
287 |         criterion = nn.MSELoss()
288 |     else:
289 |         criterion = nn.BCEWithLogitsLoss()
290 |
291 |     optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate,
292 |                                  weight_decay=weight_decay)
293 |     early_stopper = EarlyStopping(patience=patience)
294 |
295 |     history = {"train_loss": [], "valid_loss": [], "epoch_seconds": []}
296 |
297 |     for epoch in range(1, epochs + 1):
298 |         epoch_start = time.time()
299 |
300 |         model.train()
301 |         train_losses = []
302 |         for users, items, labels in train_loader:
303 |             users, items, labels = users.to(device), items.to(device), labels.to(device)
304 |
305 |             optimizer.zero_grad()
306 |             loss = criterion(model(users, items), labels)
307 |             loss.backward()
308 |             optimizer.step()
309 |             train_losses.append(loss.item())
310 |
311 |         # eval() disables dropout; no_grad() skips gradient tracking.
312 |         model.eval()
313 |         valid_losses = []
314 |         with torch.no_grad():
315 |             for users, items, labels in valid_loader:
```

```
316 |                 users, items, labels = users.to(device), items.to(device), labels.to(device)
317 |                 valid_losses.append(criterion(model(users, items), labels).item())
318 |
319 |         avg_train = float(np.mean(train_losses))
320 |         avg_valid = float(np.mean(valid_losses))
321 |         elapsed = time.time() - epoch_start
322 |
323 |         history["train_loss"].append(avg_train)
324 |         history["valid_loss"].append(avg_valid)
325 |         history["epoch_seconds"].append(elapsed)
326 |
327 |         if verbose:
328 |             print("  epoch {:02d} | train {:.4f} | valid {:.4f} | {:.1f}s".format(
329 |                 epoch, avg_train, avg_valid, elapsed))
330 |
331 |         if early_stopper.step(avg_valid, model, epoch):
332 |             if verbose:
333 |                 print("  early stopping at epoch {} (best was epoch {})".format(
334 |                     epoch, early_stopper.best_epoch))
335 |             break
336 |
337 |     if early_stopper.best_state is not None:
338 |         model.load_state_dict(early_stopper.best_state)
339 |
340 |     history["best_epoch"] = early_stopper.best_epoch
341 |     history["best_valid_loss"] = early_stopper.best_loss
342 |
343 |     return model, history
```

Standard PyTorch structure. model.train() and model.eval() toggle dropout — forgetting eval() is a classic bug that makes validation loss noisy and inflated. torch.no_grad() in the validation pass skips gradient tracking, cutting memory and time. The loss is MSE for explicit feedback and BCEWithLogits for implicit.

3.7.8 Batched inference

models/ncf_recommender.py — lines 412-443

```
412 | def score_all_items(model, user_id: int, user_to_index: dict, item_to_index: dict,
413 |                     exclude_item_ids: list = None, feedback_type: str = "explicit",
414 |                     device: str = None) -&gt; pd.Series:
415 |     """Score every candidate item for one user in a single batched pass."""
416 |     if device is None:
417 |         device = get_device()
418 |
419 |     if user_id not in user_to_index:
420 |         return pd.Series(dtype=float)
421 |
422 |     model.eval()
423 |     model.to(device)
424 |
425 |     exclude = set(exclude_item_ids or [])
426 |     candidate_ids = [iid for iid in item_to_index if iid not in exclude]
427 |
428 |     if not candidate_ids:
429 |         return pd.Series(dtype=float)
430 |
431 |     # One batched pass; per-item calls would be thousands of tiny forwards.
432 |     item_tensor = torch.tensor([item_to_index[i] for i in candidate_ids],
433 |                                dtype=torch.long, device=device)
434 |     user_tensor = torch.full((len(candidate_ids),), user_to_index[user_id],
435 |                              dtype=torch.long, device=device)
436 |
437 |     with torch.no_grad():
438 |         scores = model(user_tensor, item_tensor)
439 |         if feedback_type == "implicit":
440 |             scores = torch.sigmoid(scores)
441 |         scores = scores.cpu().numpy()
442 |
443 |     return pd.Series(scores, index=candidate_ids).sort_values(ascending=False)
```

Every candidate item is scored in a **single batched forward pass**. Scoring items one at a time would issue thousands of tiny forward passes per request; batching keeps a full catalogue scoring inside a few

milliseconds, which is what makes real-time serving viable at all.

3.7.9 Saving and loading

models/ncf_recommender.py — lines 470-493

```
470 | def save_ncf_artifacts(model, history: dict, feedback_type: str, tag: str = "ncf") -&gt; None:
471 |     """Persist weights, history and the architecture spec."""
472 |     # state_dict, not the model object: pickling the class means any later
473 |     # refactor of this file silently breaks loading.
474 |     torch.save(model.state_dict(),
475 |                os.path.join(PROCESSED_DATA_PATH, tag + "_model_" + feedback_type + ".pt"))
476 |     save_pickle(history, tag + "_history_" + feedback_type + ".pkl")
477 |
478 |     # Read architecture off the model, not the constants, so an overridden
479 |     # model reloads as what it actually is.
480 |     dropout_layers = [m.p for m in model.mlp if isinstance(m, nn.Dropout)]
481 |     hidden_dims = [m.out_features for m in model.mlp if isinstance(m, nn.Linear)]
482 |
483 |     save_pickle(
484 |         {
485 |             "num_users": model.user_embedding.num_embeddings,
486 |             "num_items": model.item_embedding.num_embeddings,
487 |             "embedding_dim": model.user_embedding.embedding_dim,
488 |             "hidden_dims": hidden_dims,
489 |             "dropout": dropout_layers[0] if dropout_layers else DROPOUT,
490 |             "feedback_type": feedback_type,
491 |         },
492 |         tag + "_architecture_" + feedback_type + ".pkl",
493 |     )
```

Save the state_dict, not the model object

Saving the whole object pickles the class definition with it, so any later refactor of this file silently breaks loading. A state_dict is just tensors; it reloads into a freshly constructed model of the same shape. The architecture spec is read back **off the model itself** rather than off the module constants, so a model trained with overridden hyperparameters reloads as what it actually is rather than what the defaults claim.

3.7.10 Regularisation for the implicit model

models/ncf_recommender.py — lines 53-53

```
53 | IMPLICIT_WEIGHT_DECAY = 1e-3
```

With 1:4 negative sampling the implicit training set is ~230k rows against ~260k embedding parameters, so with default settings the model reached its best validation loss in a **single epoch** and overfitted from epoch 2 onward. A regularisation sweep showed final quality is capacity-bound at ~0.858 accuracy regardless of settings — but weight decay 1e-3 with dropout 0.40 reaches that ceiling over ~20 epochs instead of 1, giving a genuine convergence curve and a meaningful early-stopping signal.

Configuration	Best epoch	Valid loss	Accuracy	F1
dim 32, wd 1e-5, drop 0.25	1	0.3519	0.857	0.587
dim 16, wd 1e-4, drop 0.35	2	0.3519	0.858	0.573
dim 32, wd 1e-3, drop 0.40	**11**	**0.3512**	**0.858**	0.573
dim 16, wd 1e-3, drop 0.30	5	0.3524	0.856	0.596

3.8 models/hybrid_recommender.py

411 lines — the score fusion service that both applications load.

3.8.1 The fusion weights

models/hybrid_recommender.py — lines 30-35

```
30 | HYBRID_WEIGHTS = {
31 |     "item_cf": 0.45,
32 |     "collaborative": 0.10,
33 |     "content": 0.10,
34 |     "ncf": 0.35,
35 | }
```

Weights chosen by measurement, not intuition

The first version fused SVD + content + NCF at 0.35 / 0.25 / 0.40 and scored NDCG@10 = 0.0947 — **worse than plain item-based CF on its own (0.1245)**. The hybrid was blending three mediocre signals while ignoring the strongest one available. The weights below were selected by a sweep over a 250-customer cohort.

Configuration	NDCG@10
item-based CF alone	0.1245
svd + content + ncf (original)	0.0947
item_cf + content + ncf	0.1282
item_cf .50 / content .10 / ncf .40	0.1276
item_cf .45 / svd .10 / content .10 / ncf .35 — selected	**0.1293**
item_cf .70 / ncf .30 (no content)	0.1181

SVD is retained at low weight because it generalises to customers whose exact co-rating neighbours are absent. Content is weak alone (0.0116) but has by far the best catalogue coverage (75.4%), so a small weight buys reach without materially costing accuracy. Dropping either loses NDCG.

3.8.2 Loading artifacts once

models/hybrid_recommender.py — lines 87-129

```
 87 |     @classmethod
 88 |     def load(cls, feedback_type: str = "implicit") -&gt; "HybridRecommender":
 89 |         """Load every artifact from data/processed."""
 90 |         # Defaults to implicit: the explicit model predicts ratings well
 91 |         # (RMSE 0.91) but scored NDCG@10 = 0.0000 for ranking.
 92 |         users_df, items_df, interactions_df = load_all()
 93 |
 94 |         user_item_matrix = load_pickle("user_item_matrix.pkl")
 95 |         predicted_ratings = load_pickle("predicted_ratings.pkl")
 96 |         item_popularity = load_pickle("item_popularity_features.pkl")
 97 |         content_similarity = load_pickle("content_similarity.pkl")
 98 |         item_similarity = load_pickle("item_similarity.pkl")
 99 |         tfidf_matrix = load_pickle("tfidf_matrix.pkl")
100 |         tfidf_vectorizer = load_pickle("tfidf_vectorizer.pkl")
101 |
102 |         # NCF is optional but its absence must be loud: a previous revision
103 |         # swallowed this and served with the deep signal silently at zero.
104 |         try:
105 |             ncf_model = load_ncf_model(feedback_type=feedback_type)
106 |             user_to_index = load_pickle("ncf_user_to_index.pkl")
107 |             item_to_index = load_pickle("ncf_item_to_index.pkl")
108 |         except FileNotFoundError as exc:
109 |             print("WARNING: NCF artifacts unavailable ({}).".format(exc))
110 |             print("         Running on collaborative + content signals only.")
111 |             print("         Train with:  python models/ncf_recommender.py")
112 |             ncf_model, user_to_index, item_to_index = None, {}, {}
113 |
114 |         return cls(
115 |             users_df=users_df,
116 |             items_df=items_df,
117 |             interactions_df=interactions_df,
118 |             user_item_matrix=user_item_matrix,
119 |             predicted_ratings=predicted_ratings,
120 |             popularity_lookup=build_popularity_lookup(item_popularity),
121 |             content_similarity=content_similarity,
122 |             item_similarity=item_similarity,
123 |             tfidf_matrix=tfidf_matrix,
124 |             tfidf_vectorizer=tfidf_vectorizer,
125 |             ncf_model=ncf_model,
126 |             user_to_index=user_to_index,
127 |             item_to_index=item_to_index,
128 |             feedback_type=feedback_type,
129 |         )
```

Failing loudly

The NCF signal is optional at load time, **but its absence must be loud**. The previous revision swallowed this failure in a bare try/except and served recommendations with the deep-learning weight silently set to zero. The API reported healthy the entire time. Now a missing model prints the exact file path and the command to fix it.

3.8.3 Precomputing user history

Each customer's interaction set is computed **once at load** into a dictionary. Filtering the full 138,345-row interaction log per request is the single biggest avoidable cost in the serving path, and it would be paid on every call.

3.8.4 The four signals

models/hybrid_recommender.py — lines 155-172

```
155 |     def item_cf_scores(self, user_id: int) -&gt; pd.Series:
156 |         """Item-based CF scores - the strongest single signal (NDCG 0.1245)."""
157 |         if self.item_similarity is None or user_id not in self.user_item_matrix.index:
158 |             return pd.Series(dtype=float)
159 |
160 |         user_ratings = self.user_item_matrix.loc[user_id]
161 |         rated = user_ratings[user_ratings &gt; 0]
162 |
163 |         if rated.empty:
164 |             return pd.Series(dtype=float)
165 |
166 |         scores = pd.Series(
167 |             self.item_similarity[rated.index].to_numpy() @ rated.to_numpy(),
168 |             index=self.item_similarity.index,
169 |         )
170 |         scores = scores.drop(list(self.seen_items(user_id)), errors="ignore")
171 |
172 |         return min_max_normalize(scores)
```

Item-based CF is the strongest single signal (NDCG@10 = 0.1245 standalone against SVD's 0.0462), which is why it carries the largest fusion weight.

models/hybrid_recommender.py — lines 184-208

```
184 |     def content_scores(self, user_id: int) -&gt; pd.Series:
185 |         """Content similarity between the user's taste profile and each item."""
186 |         user_rows = self.interactions_df.loc[self.interactions_df["user_id"] == user_id]
187 |         if user_rows.empty:
188 |             return pd.Series(dtype=float)
189 |
190 |         liked_mask = (
191 |             (user_rows["rating"] &gt;= RELEVANCE_RATING_THRESHOLD)
192 |             | (user_rows["purchase"] == 1)
193 |         ).fillna(False)
194 |
195 |         liked_items = [i for i in user_rows.loc[liked_mask, "item_id"].unique()
196 |                        if i in self.content_similarity.index]
197 |
198 |         if not liked_items:
199 |             return pd.Series(dtype=float)
200 |
201 |         # Mean of liked items' similarity rows - equivalent to a TF-IDF
202 |         # centroid but avoids a sparse matmul per request.
203 |         profile_similarity = self.content_similarity.loc[liked_items].mean(axis=0)
204 |         profile_similarity = profile_similarity.drop(
205 |             list(self.seen_items(user_id)), errors="ignore"
206 |         )
207 |
208 |         return min_max_normalize(profile_similarity)
```

Content scores are built from the **item-item** similarity matrix rather than by re-running cosine similarity against the TF-IDF matrix. The customer's profile is the mean of their liked items' similarity rows, which is mathematically equivalent and avoids a sparse matrix multiply on every request.

3.8.5 Weight renormalisation

models/hybrid_recommender.py — lines 229-270

```
229 |     def fuse(self, item_cf, collaborative, content, ncf) -&gt; pd.DataFrame:
230 |         """Weighted linear fusion, then popularity-balanced re-ranking."""
231 |         available = {
232 |             "item_cf": item_cf,
233 |             "collaborative": collaborative,
234 |             "content": content,
235 |             "ncf": ncf,
236 |         }
237 |         active = {name: s for name, s in available.items() if not s.empty}
238 |
239 |         if not active:
240 |             return pd.DataFrame()
241 |
242 |         # Renormalise over active signals, otherwise a user the NCF has never
243 |         # seen gets scores scaled by the surviving weight mass and they stop
244 |         # being comparable across users.
245 |         weight_mass = sum(self.weights[name] for name in active)
246 |         effective_weights = {name: self.weights[name] / weight_mass for name in active}
247 |
248 |         all_item_ids = sorted(set().union(*(set(s.index) for s in active.values())))
249 |         fused = pd.DataFrame(index=all_item_ids)
250 |
251 |         for name, series in available.items():
252 |             fused[name + "_score"] = series.reindex(fused.index).fillna(0.0)
253 |
254 |         fused["hybrid_raw_score"] = sum(
255 |             effective_weights[name] * fused[name + "_score"] for name in active
256 |         )
257 |
258 |         reranked = rerank_with_popularity_balance(
259 |             fused["hybrid_raw_score"],
260 |             self.popularity_lookup,
261 |             candidate_pool=CANDIDATE_POOL_SIZE,
262 |         )
263 |
264 |         fused = fused.loc[reranked.index].join(
265 |             reranked[["adjusted_score", "interaction_count", "is_long_tail",
266 |                       "popularity_penalty", "long_tail_boost"]]
267 |         )
268 |
269 |         fused["active_signals"] = ", ".join(sorted(active))
270 |         return fused.sort_values("adjusted_score", ascending=False)
```

Why renormalise over active signals

If a customer is unknown to the NCF, that signal produces nothing. Without renormalisation their final score would silently be multiplied by the surviving weight mass (0.65). That does not change their *ranking*, but it makes scores **incomparable across customers** and meaningless to display in a UI or an API response.

3.8.6 Cold-start fallback

models/hybrid_recommender.py — lines 275-302

```
275 |     def cold_start_recommendations(self, user_id: int, top_n: int = 10) -&gt; pd.DataFrame:
276 |         """Two-tier fallback: registration profile, else global popularity."""
277 |         user_row = self.users_df.loc[self.users_df["user_id"] == user_id]
278 |
279 |         if user_row.empty:
280 |             ranked = rerank_with_popularity_balance(
281 |                 self.popularity_lookup["popularity_ratio"],
282 |                 self.popularity_lookup,
283 |                 candidate_pool=CANDIDATE_POOL_SIZE,
284 |             ).head(top_n)
285 |
286 |             result = self.items_df.loc[self.items_df["item_id"].isin(ranked.index)].copy()
287 |             result["hybrid_score"] = ranked["adjusted_score"].reindex(
288 |                 result["item_id"]).to_numpy()
289 |             result["strategy"] = STRATEGY_UNKNOWN_USER
290 |             return result.sort_values("hybrid_score", ascending=False)
291 |
292 |         profile = user_row.iloc[0]
293 |         result = recommend_for_new_user_by_profile(
294 |             preferred_category=profile["preferred_category"],
295 |             items_df=self.items_df,
296 |             user_segment=profile["user_segment"],
297 |             n_top=top_n,
298 |         ).copy()
299 |
300 |         result = result.rename(columns={"cold_start_score": "hybrid_score"})
301 |         result["strategy"] = STRATEGY_COLD_USER
302 |         return result
```

Two tiers. A known-but-inactive customer is served from their registration profile — declared category plus segment price band. A genuinely unknown customer gets de-biased global popularity, because there is nothing else to personalise on.

models/hybrid_recommender.py — lines 307-337

```
307 |     def recommend(self, user_id: int, top_n: int = 10) -&gt; pd.DataFrame:
308 |         """Top-N recommendations with per-signal attribution and a strategy label."""
309 |         if self.is_cold_start_user(user_id):
310 |             return self.cold_start_recommendations(user_id, top_n)
311 |
312 |         fused = self.fuse(
313 |             self.item_cf_scores(user_id),
314 |             self.collaborative_scores(user_id),
315 |             self.content_scores(user_id),
316 |             self.ncf_scores(user_id),
317 |         )
318 |
319 |         if fused.empty:
320 |             return self.cold_start_recommendations(user_id, top_n)
321 |
322 |         top = fused.head(top_n).copy()
323 |         top.index.name = "item_id"
324 |         top = top.reset_index()
325 |
326 |         catalogue_columns = [
327 |             c for c in ["item_id", "title", "category", "subcategory", "brand",
328 |                         "price", "content_tags"]
329 |             if c in self.items_df.columns
330 |         ]
331 |         top = top.merge(self.items_df[catalogue_columns], on="item_id", how="left")
332 |
333 |         top = top.rename(columns={"adjusted_score": "hybrid_score"})
334 |         # Always label the strategy so a caller can tell personalised from fallback.
335 |         top["strategy"] = STRATEGY_HYBRID
336 |
337 |         return top
```

Note is_cold_start_user is a **threshold**, not a binary has-any-history check. A customer with a single click carries almost no signal, and treating them as warm produces confidently wrong recommendations from one data point.

Every returned frame carries a strategy column. A caller must always be able to distinguish a personalised result from a fallback — presenting the two identically is how a dashboard ends up claiming a brand-new customer has a learned taste profile.

3.9 models/explainability.py

397 lines — the three mandated explanation types.

3.9.1 Signal attribution

models/explainability.py — lines 47-67

```
47 | def attribute_signals(recommendation_row: pd.Series, weights: dict) -&gt; pd.DataFrame:
48 |     """Decompose a hybrid score into each signal's actual contribution."""
49 |     # Report weight x score, not the raw score: a high score on a low-weight
50 |     # signal contributes little to the ranking.
51 |     rows = []
52 |
53 |     for column, weight_key in SIGNAL_WEIGHT_KEYS.items():
54 |         score = float(recommendation_row.get(column, 0.0) or 0.0)
55 |         weight = float(weights.get(weight_key, 0.0))
56 |         rows.append({
57 |             "signal": SIGNAL_LABELS[column],
58 |             "raw_score": score,
59 |             "weight": weight,
60 |             "contribution": score * weight,
61 |         })
62 |
63 |     frame = pd.DataFrame(rows)
64 |     total = frame["contribution"].sum()
65 |     frame["contribution_share"] = frame["contribution"] / total if total &gt; 0 else 0.0
66 |
67 |     return frame.sort_values("contribution", ascending=False).reset_index(drop=True)
```

Reporting raw per-signal scores would mislead: a signal with a high score but a low fusion weight contributes little to the final ranking. What matters for explanation is **weight × score** — the share of the decision each signal is genuinely responsible for.

3.9.2 Building the explanation

models/explainability.py — lines 84-137

```
 84 | def explain_recommendation(recommender, user_id: int, recommendation_row: pd.Series) -&gt; str:
 85 |     """Build a human-readable justification for one recommended item."""
 86 |     # Every clause is conditional on a measured quantity - never a template.
 87 |     item_id = int(recommendation_row["item_id"])
 88 |     strategy = recommendation_row.get("strategy", "")
 89 |
 90 |     item = recommender.item_details(item_id)
 91 |     user = recommender.user_details(user_id)
 92 |
 93 |     if item is None:
 94 |         return "Recommended by the hybrid ranking model."
 95 |
 96 |     if strategy == "cold_start_user_profile":
 97 |         segment = user["user_segment"] if user is not None else "your"
 98 |         return (
 99 |             f"You are new here, so this is based on your registration profile rather than "
100 |             f"purchase history: it sits in {item['category']}, the category you selected, "
101 |             f"and is priced at INR {item['price']:,.0f}, which fits the typical "
102 |             f"{segment} budget. Recommendations will become personalised once you have "
103 |             f"browsed a few items."
104 |         )
105 |
106 |     if strategy == "global_popularity_fallback":
107 |         return (
108 |             f"We have no profile for this account yet, so this shows our best-performing "
109 |             f"{item['category']} item across all customers, adjusted so that popular "
110 |             f"products do not crowd out everything else."
111 |         )
112 |
113 |     reasons = []
114 |
115 |     signal_label, share = dominant_signal(recommendation_row, recommender.weights)
116 |     for column, meaning in SIGNAL_MEANING.items():
117 |         if SIGNAL_LABELS.get(column) == signal_label:
118 |             reasons.append(f"{meaning} (driving {share:.0%} of this recommendation)")
119 |             break
120 |
121 |     for column, meaning in SIGNAL_MEANING.items():
122 |         if SIGNAL_LABELS.get(column) == signal_label:
123 |             continue
124 |         if float(recommendation_row.get(column, 0.0) or 0.0) &gt; 0.65:
125 |             reasons.append(meaning)
126 |
127 |     if user is not None and item["category"] == user["preferred_category"]:
128 |         reasons.append(f"it is in {item['category']}, your stated category of interest")
129 |
130 |     # Long-tail promotion is disclosed, not hidden.
131 |     if recommendation_row.get("is_long_tail", 0) == 1:
132 |         reasons.append("it is an under-exposed product we are surfacing for variety")
133 |
134 |     if not reasons:
135 |         return "Recommended by the combined collaborative, content, and deep learning ranking."
136 |
137 |     return "Recommended because " + "; ".join(reasons) + "."
```

The design principle

Every clause is conditional on a real measured quantity. If a signal did not fire, its clause is not emitted. A cold-start customer therefore gets a visibly different and *honest* explanation rather than the same confident sentence with different nouns. A generated sentence that does not correspond to the real reason is worse than no explanation at all — it manufactures false confidence and will eventually contradict the model in front of a customer.

Long-tail promotion is **disclosed rather than hidden**. If the ranking was adjusted for catalogue diversity, the customer is told so.

3.9.3 Evidence from the customer's own history

models/explainability.py — lines 143-170

```
143 | def explain_via_similar_items(recommender, user_id: int, item_id: int,
144 |                               n_evidence: int = 3) -&gt; pd.DataFrame:
145 |     """Show which of the user's own purchases most resemble this suggestion."""
146 |     seen = recommender.seen_items(user_id)
147 |
148 |     if not seen or item_id not in recommender.content_similarity.index:
149 |         return pd.DataFrame(columns=["item_id", "title", "category", "similarity"])
150 |
151 |     known = [i for i in seen if i in recommender.content_similarity.columns]
152 |     if not known:
153 |         return pd.DataFrame(columns=["item_id", "title", "category", "similarity"])
154 |
155 |     similarities = recommender.content_similarity.loc[item_id, known].sort_values(ascending=False)
156 |     top = similarities.head(n_evidence)
157 |
158 |     evidence = []
159 |     for evidence_item_id, similarity in top.items():
160 |         details = recommender.item_details(int(evidence_item_id))
161 |         if details is None:
162 |             continue
163 |         evidence.append({
164 |             "item_id": int(evidence_item_id),
165 |             "title": details["title"],
166 |             "category": details["category"],
167 |             "similarity": round(float(similarity), 4),
168 |         })
169 |
170 |     return pd.DataFrame(evidence)
```

This is the most persuasive evidence available, because it is grounded entirely in the customer's own behaviour. 'Because you bought X' is verifiable by the customer in a way that 'because the model said so' never is.

3.9.4 An honest tension

models/explainability.py — lines 212-235

```
212 | def render_similar_user_evidence(evidence: dict) -&gt; str:
213 |     """Phrase the collaborative evidence, including when there is none."""
214 |     # Empty evidence is expected: the re-ranker promotes long-tail items,
215 |     # which are long-tail precisely because few people interacted with them.
216 |     considered = evidence.get("neighbours_considered", 0)
217 |     rated = evidence.get("neighbours_who_rated", 0)
218 |     liked = evidence.get("neighbours_who_liked", 0)
219 |     mean_rating = evidence.get("mean_rating")
220 |
221 |     if considered == 0:
222 |         return "No comparable customers available for this account yet."
223 |
224 |     if rated == 0:
225 |         return (
226 |             f"None of your {considered} closest customer matches have rated this item - "
227 |             f"it is a low-exposure product surfaced for catalogue variety, so the "
228 |             f"recommendation rests on content and model signals rather than crowd evidence."
229 |         )
230 |
231 |     return (
232 |         f"{liked} of the {rated} closest customer matches who rated this item scored it "
233 |         f"{RELEVANCE_RATING_THRESHOLD} or above (mean {mean_rating}), out of {considered} "
234 |         f"neighbours considered."
235 |     )
```

The diversity mechanism undermines its own evidence

Similar-customer evidence is often empty, and the reason is **structural rather than a defect**: the re-ranker deliberately promotes under-exposed long-tail items, and an item is long-tail precisely because few people have interacted with it. The system states this plainly instead of hiding it. A customer told 'customers like you loved this' about an item no comparable customer has ever rated is being misled.

The neighbourhood is also deliberately wide (50). At 99% matrix sparsity a 5-user neighbourhood overlaps on any specific item almost never, so the evidence came back empty for nearly every recommendation and told the customer nothing.

3.9.5 Content justification

models/explainability.py — lines 241-280

```
241 | def explain_content_similarity(recommender, source_item_id: int,
242 |                                recommended_item_id: int, n_terms: int = 5) -&gt; dict:
243 |     """Expose the TF-IDF terms and attributes linking two items."""
244 |     source = recommender.item_details(source_item_id)
245 |     recommended = recommender.item_details(recommended_item_id)
246 |
247 |     if source is None or recommended is None:
248 |         return {"shared_terms": [], "similarity": None, "shared_attributes": []}
249 |
250 |     similarity = None
251 |     if (source_item_id in recommender.content_similarity.index
252 |             and recommended_item_id in recommender.content_similarity.columns):
253 |         similarity = round(
254 |             float(recommender.content_similarity.loc[source_item_id, recommended_item_id]), 4
255 |         )
256 |
257 |     shared_terms = top_shared_terms(
258 |         source_item_id,
259 |         recommended_item_id,
260 |         recommender.items_df,
261 |         recommender.tfidf_vectorizer,
262 |         recommender.tfidf_matrix,
263 |         n_terms=n_terms,
264 |     )
265 |
266 |     shared_attributes = []
267 |     for field in ("category", "subcategory", "brand"):
268 |         if field in source and field in recommended and source[field] == recommended[field]:
269 |             shared_attributes.append(f"{field.replace('_', ' ')}: {source[field]}")
270 |
271 |     source_tags = set(str(source.get("content_tags", "")).split("|"))
272 |     recommended_tags = set(str(recommended.get("content_tags", "")).split("|"))
273 |     shared_tags = sorted(t for t in (source_tags &amp;amp; recommended_tags) if t)
274 |
275 |     return {
276 |         "similarity": similarity,
277 |         "shared_terms": shared_terms,
278 |         "shared_attributes": shared_attributes,
279 |         "shared_tags": shared_tags,
280 |     }
```

Returns the specific TF-IDF terms, the shared structured attributes, and the overlapping content tags. This is what converts 'similarity 0.83' into something a merchandiser can audit.

3.10 src/evaluation_metrics.py

198 lines — ranking metrics written from first principles.

3.10.1 Precision@K and Recall@K

src/evaluation_metrics.py — lines 10-20

```
10 | def precision_at_k(recommended: list, relevant: set, k: int) -&gt; float:
11 |     """Fraction of the top-K recommendations that were relevant."""
12 |     if not recommended or not relevant:
13 |         return 0.0
14 |
15 |     # Denominator is len(top_k), not k: do not penalise a model for
16 |     # candidates it was never able to return.
17 |     top_k = recommended[:k]
18 |     hits = sum(1 for item in top_k if item in relevant)
19 |
20 |     return hits / len(top_k)
```

Note the Precision denominator: len(top_k), not k. If the model could only produce 6 candidates, scoring it out of 10 penalises it for items it was never able to return.

Recall has a ceiling worth understanding: if a customer has 40 relevant items in the test period, Recall@10 **cannot exceed 0.25** no matter how perfect the model is. Recall should always be read alongside the relevant-item count per customer.

3.10.2 MAP@K

src/evaluation_metrics.py — lines 32-48

```
32 | def average_precision_at_k(recommended: list, relevant: set, k: int) -&gt; float:
33 |     """Average precision for one user, recomputed at each hit position."""
34 |     if not recommended or not relevant:
35 |         return 0.0
36 |
37 |     hits = 0
38 |     precision_sum = 0.0
39 |
40 |     for position, item in enumerate(recommended[:k], start=1):
41 |         if item in relevant:
42 |             hits += 1
43 |             precision_sum += hits / position
44 |
45 |     # Normalise by min(relevant, k), not hits: dividing by hits alone would
46 |     # score 1.0 for finding a single relevant item.
47 |     denominator = min(len(relevant), k)
48 |     return precision_sum / denominator if denominator &gt; 0 else 0.0
```

Normalised by min(len(relevant), k) rather than by the hit count. Dividing by hits alone would award a perfect 1.0 to a model that found exactly one relevant item and placed it first, which is plainly not perfect performance.

3.10.3 NDCG@K

src/evaluation_metrics.py — lines 51-77

```
51 | def ndcg_at_k(recommended: list, relevant: set, k: int,
52 |               relevance_scores: dict = None) -&gt; float:
53 |     """Normalised discounted cumulative gain, with optional graded relevance."""
54 |     if not recommended or not relevant:
55 |         return 0.0
56 |
57 |     def gain(item) -&gt; float:
58 |         if relevance_scores is None:
59 |             return 1.0 if item in relevant else 0.0
60 |         return float(relevance_scores.get(item, 0.0))
61 |
62 |     dcg = sum(
63 |         gain(item) / np.log2(position + 1)
64 |         for position, item in enumerate(recommended[:k], start=1)
65 |         if item in relevant
66 |     )
67 |
68 |     if relevance_scores is None:
69 |         ideal_gains = [1.0] * min(len(relevant), k)
70 |     else:
71 |         ideal_gains = sorted(
72 |             (relevance_scores.get(i, 0.0) for i in relevant), reverse=True
73 |         )[:k]
74 |
75 |     idcg = sum(g / np.log2(position + 1) for position, g in enumerate(ideal_gains, start=1))
76 |
77 |     return dcg / idcg if idcg &gt; 0 else 0.0
```

The most important metric here. Precision@10 treats a hit at position 1 and a hit at position 10 identically; on a storefront those are worth very different amounts, because click-through decays sharply with position. NDCG applies a logarithmic positional discount and is therefore the metric that best tracks revenue.

Graded relevance is supported, so a 5-star hit outranks a 4-star one. The ideal DCG is computed from the best achievable ordering *given what the customer actually found relevant*, so a customer with only 2 relevant items is not penalised for failing to fill 10 slots.

3.10.4 Beyond-accuracy metrics

src/evaluation_metrics.py — lines 90-99

```
90 | def catalogue_coverage(all_recommendations: list, total_items: int) -&gt; float:
91 |     """Share of the catalogue that ever appears in any recommendation."""
92 |     if total_items == 0:
93 |         return 0.0
94 |
95 |     recommended_items = set()
96 |     for recommendations in all_recommendations:
97 |         recommended_items.update(recommendations)
98 |
99 |     return len(recommended_items) / total_items
```

Why coverage is a first-class metric

A model can post excellent precision while only ever recommending 32 items out of 2,500. Commercially that is a **failure** — the remaining inventory is invisible and will never sell. Accuracy metrics alone cannot detect this. The popularity baseline has 1.3% coverage, and so did the broken NCF.

src/evaluation_metrics.py — lines 115-124

```
115 | def intra_list_diversity(recommendations: list, similarity_df: pd.DataFrame) -&gt; float:
116 |     """Average pairwise dissimilarity within one recommendation list."""
117 |     valid = [i for i in recommendations if i in similarity_df.index]
118 |     if len(valid) &lt; 2:
119 |         return 0.0
120 |
121 |     submatrix = similarity_df.loc[valid, valid].to_numpy()
122 |     upper_triangle = submatrix[np.triu_indices(len(valid), k=1)]
123 |
124 |     return float(1.0 - upper_triangle.mean())
```

Intra-list diversity guards against the classic failure where a model returns ten near-identical products; each is individually relevant, so precision looks excellent, but the list gives the customer no real choice. **Novelty** is the mean self-information of recommended items — a recommender that only returns best-sellers scores near zero and is, in effect, an expensive way to reproduce a top-sellers list.

3.11 notebooks/recommendation_evaluation.py

462 lines — the benchmark. This is where the two biggest bugs were found.

3.11.1 Which NCF variant to rank with

notebooks/recommendation_evaluation.py — lines 57-57

```
57 | NCF_RANKING_MODE = "implicit"
```

This comment block documents the single most important modelling lesson in the project, and it is worth reading in full in the source.

3.11.2 Ground truth

notebooks/recommendation_evaluation.py — lines 63-86

```
63 | def build_ground_truth(test_df):
64 |     """Extract what each user actually found relevant during the test period."""
65 |     relevant_by_user = {}
66 |     graded_by_user = {}
67 |
68 |     is_relevant = (
69 |         (test_df["rating"] &gt;= RELEVANCE_RATING_THRESHOLD).fillna(False)
70 |         | (test_df["purchase"] == 1)
71 |     )
72 |     relevant_rows = test_df.loc[is_relevant]
73 |
74 |     for user_id, group in relevant_rows.groupby("user_id"):
75 |         relevant_by_user[int(user_id)] = set(group["item_id"].tolist())
76 |
77 |         # A purchase without a rating still counts; give it the threshold value
78 |         # so it contributes real gain without outranking an explicit 5-star.
79 |         grades = {}
80 |         for item_id, rating in zip(group["item_id"], group["rating"]):
81 |             grades[int(item_id)] = (
82 |                 float(rating) if pd.notna(rating) else float(RELEVANCE_RATING_THRESHOLD)
83 |             )
84 |         graded_by_user[int(user_id)] = grades
85 |
86 |     return relevant_by_user, graded_by_user
```

An unrated purchase is graded at the threshold value — it contributes real gain to NDCG without outranking an explicit 5-star rating.

3.11.3 Enforcing no leakage

notebooks/recommendation_evaluation.py — lines 92-126

```
 92 | def build_train_artifacts(train_df, items_df):
 93 |     """Rebuild every model from the training period alone."""
 94 |     print("  rebuilding models on the training period only ...")
 95 |
 96 |     user_item_matrix = create_user_item_matrix(train_df)
 97 |     normalized_matrix = create_normalized_user_item_matrix(train_df)
 98 |     implicit_matrix = create_implicit_feedback_matrix(train_df)
 99 |     item_popularity = create_item_popularity_features(train_df, items_df)
100 |
101 |     popularity_lookup = build_popularity_lookup(item_popularity)
102 |
103 |     svd, user_factors, item_factors = create_svd_model(normalized_matrix)
104 |     predicted_ratings = create_predicted_rating_matrix(
105 |         normalized_matrix, user_factors, item_factors
106 |     )
107 |
108 |     item_similarity = pd.DataFrame(
109 |         cosine_similarity(user_item_matrix.T).astype(np.float32),
110 |         index=user_item_matrix.columns,
111 |         columns=user_item_matrix.columns,
112 |     )
113 |
114 |     popularity_ranking = (
115 |         item_popularity.sort_values("interaction_count", ascending=False)["item_id"].tolist()
116 |     )
117 |
118 |     return {
119 |         "user_item_matrix": user_item_matrix,
120 |         "implicit_matrix": implicit_matrix,
121 |         "item_popularity": item_popularity,
122 |         "popularity_lookup": popularity_lookup,
123 |         "predicted_ratings": predicted_ratings,
124 |         "item_similarity": item_similarity,
125 |         "popularity_ranking": popularity_ranking,
126 |     }
```

This is the step that actually enforces no-leakage

Reusing the artifacts in data/processed/ would have been far quicker, but those were fitted on the **full history including the test window**. Every metric computed against them would be inflated. The models must be refitted on train-only data, and that is what this function does.

The split is verified with assertions rather than assumed. If a future change ever breaks the temporal separation, the benchmark crashes rather than silently reporting inflated numbers.

3.11.4 Fair comparison

Every model is called through the same interface, on the same customers, with the same exclusion set. Customers with no relevant test item are excluded — including them would drag every metric toward zero by an amount that depends on the split rather than on model quality, making runs incomparable.

3.12 app/app_fastapi.py

465 lines — the REST service. Transport only; no scoring logic.

3.12.1 Response models

app/app_fastapi.py — lines 66-76

```
66 | class RecommendationResponse(BaseModel):
67 |     user_id: int
68 |     strategy: str = Field(..., description="hybrid_fusion | cold_start_user_profile | global_popularity_fallback")
69 |     top_n: int
70 |     user_segment: str | None = None
71 |     interaction_count: int
72 |     recommendations: list[RecommendationItem]
73 |
74 |     # Business context: what this recommendation set is worth if it converts.
75 |     total_basket_value: float = Field(..., description="Sum of recommended item prices, INR")
76 |     average_price: float
```

Pydantic models are the API contract, and they also perform **output** validation. If a scoring change ever starts emitting nulls where a price is expected, the response fails here rather than silently reaching the storefront.

3.12.2 Loading once at startup

app/app_fastapi.py — lines 194-216

```
194 | @asynccontextmanager
195 | async def lifespan(app: FastAPI):
196 |     """Load all model artifacts once at startup."""
197 |     global recommender
198 |
199 |     print("Loading recommendation artifacts ...")
200 |     try:
201 |         recommender = HybridRecommender.load()
202 |         print("  users        : {:,}".format(len(recommender.users_df)))
203 |         print("  items        : {:,}".format(len(recommender.items_df)))
204 |         print("  interactions : {:,}".format(len(recommender.interactions_df)))
205 |         print("  NCF loaded   : {}".format(recommender.ncf_model is not None))
206 |         print("Service ready.")
207 |     except FileNotFoundError as exc:
208 |         # Do not start pretending to be healthy. A recommender missing its
209 |         # artifacts should refuse requests, not serve degraded results silently.
210 |         print("FAILED to load artifacts: {}".format(exc))
211 |         print("Run the pipeline first - see README Quickstart.")
212 |         recommender = None
213 |
214 |     yield
215 |
216 |     print("Shutting down recommendation service.")
```

The lifespan context manager loads all artifacts once when the service starts. Loading per request would put response times in the tens of seconds. If core artifacts are missing the service refuses to pretend it is healthy.

3.12.3 The recommend endpoint

app/app_fastapi.py — lines 270-303

```
@app.get("/recommend/{user_id}", response_model=RecommendationResponse)
def recommend(
    user_id: int,
    top_n: int = Query(10, ge=1, le=MAX_TOP_N, description="Number of items to return"),
    explain: bool = Query(True, description="Include per-signal score attribution"),
):
    """Personalised recommendations for a customer."""
    require_ready()

    if user_id &lt; 1:
        raise HTTPException(status_code=422, detail="user_id must be a positive integer.")

    frame = recommender.recommend(user_id, top_n=top_n)

    if frame.empty:
        raise HTTPException(
            status_code=404,
            detail="No recommendations could be generated for user {}.".format(user_id),
        )

    items = build_recommendation_items(user_id, frame, include_breakdown=explain)
    profile = recommender.user_details(user_id)
    prices = [item.price for item in items]

    return RecommendationResponse(
        user_id=user_id,
        strategy=str(frame["strategy"].iloc[0]),
        top_n=len(items),
        user_segment=safe_str(profile["user_segment"]) if profile is not None else None,
        interaction_count=recommender.user_interaction_count(user_id),
        recommendations=items,
        total_basket_value=round(sum(prices), 2),
        average_price=round(sum(prices) / len(prices), 2) if prices else 0.0,
    )
```

Query(10, ge=1, le=50) gives declarative validation — a request for 999 items returns HTTP 422 automatically, with no hand-written check. Every response includes the strategy field and business context: total basket value and average price point, so the caller can see what the recommendation set is worth if it converts.

3.12.4 Full endpoint list

Endpoint	Purpose
GET /	service metadata
GET /health	readiness, and whether NCF actually loaded
GET /recommend/{user_id}	top-N with explanations and signal breakdown
GET /similar-items/{item_id}	content-similar products
GET /explain/{user_id}/{item_id}	full three-part explanation
GET /users/{user_id}	customer profile and cold-start status
GET /items/{item_id}	product detail and long-tail flag
POST /recommend/batch	many customers in one call, for offline jobs

3.13 app/streamlit_recommendation_dashboard.py

551 lines — the operator console.

3.13.1 Caching the recommender

app/streamlit_recommendation_dashboard.py — lines 50-53

```
50 | @st.cache_resource(show_spinner="Loading recommendation models ...")
51 | def load_recommender():
52 |     """Load the recommender once per session."""
53 |     return HybridRecommender.load()
```

@st.cache_resource rather than @st.cache_data. This object holds several hundred megabytes of similarity matrices and a PyTorch model, none of which is serialisable in the way cache_data expects. Streamlit re-runs the entire script on every interaction, so without caching the model would reload on each click.

3.13.2 Meaningful customer cohorts

A usability decision that matters for the demo

Offering cohorts rather than a raw 6,000-entry dropdown is deliberate. Picking a customer at random almost always lands on a light user, which makes the personalisation look far weaker than it is. The cohorts let a reviewer jump straight to a highly active customer, or straight to a cold-start one.

3.13.3 The five tabs

Tab	Shows	Rubric criterion
Recommendations	ranked items, basket value, category mix, signal contribution	business recommendation display
Explainability	signal attribution chart, evidence, content justification	explainability visualisation
Similar Items	content neighbours with cold-item warning	UI usability
Cold-Start Demo	three scenarios: new customer, unknown, new product	cold-start handling
Customer History	purchases, conversion, category distribution	UI clarity

3.13.4 Honest strategy labelling

app/streamlit_recommendation_dashboard.py — lines 40-44

```
40 | STRATEGY_LABELS = {
41 |     STRATEGY_HYBRID: ("Personalised (hybrid fusion)", "success"),
42 |     STRATEGY_COLD_USER: ("Cold start - registration profile", "warning"),
43 |     STRATEGY_UNKNOWN_USER: ("Unknown customer - global fallback", "error"),
44 | }
```

The strategy label is surfaced with colour severity — green for personalised, amber for cold start, red for unknown customer. The reviewer can see at a glance which path produced the result, which is the honest presentation.

Part 4 — Results and Evaluation

4.1 Methodology

Aspect	Choice	Why
Split	final 90 days held out	a random split leaks the future and inflates every metric
Train period	2025-01-07 to 2026-04-01 (96,656 events)	everything before the cutoff
Test period	2026-04-02 to 2026-06-30 (41,689 events)	strictly later, never seen
Leakage control	models refitted on train only, asserted in code	stored artifacts were fitted on full history
Relevance	rated ≥ 4 OR purchased	an unrated purchase is still unambiguous evidence
Cohort	800 customers with ≥1 relevant test item	customers with nothing to find make runs incomparable

4.2 Ranking metrics at K = 10

Model	Precision	Recall	MAP	NDCG	Hit Rate
Hybrid	**0.0628**	**0.1372**	**0.0593**	**0.1095**	**0.3850**
Item-based CF	0.0579	0.1241	0.0593	0.1044	0.3575
NCF	0.0375	0.0858	0.0349	0.0663	0.2812
Popularity	0.0375	0.0843	0.0336	0.0645	0.2825
SVD	0.0278	0.0511	0.0230	0.0462	0.2050
Content (TF-IDF)	0.0073	0.0161	0.0049	0.0116	0.0675

4.3 Beyond-accuracy metrics

Model	Coverage	Long-tail share	Novelty	Intra-list diversity
Content (TF-IDF)	**0.754**	0.814	**12.32**	0.707
Item-based CF	0.222	0.056	7.67	0.936
Hybrid	0.202	0.040	7.91	0.911
SVD	0.096	0.069	8.57	0.955
Popularity	0.013	0.000	6.83	0.962
NCF	0.013	0.000	6.91	0.952

The content model is weakest on accuracy and strongest on coverage by a factor of three. That is precisely why it earns a place in the hybrid despite its standalone score — it is the only component that can reach the long tail at all.

4.4 Deep learning model metrics

Variant	Metric	Value	Training time
Explicit (rating prediction)	RMSE	0.9108	30 s / 17 epochs
Explicit	MAE	0.7449	
Implicit (engagement)	Accuracy	0.8583	464 s / 20 epochs
Implicit	Precision	0.7147	
Implicit	Recall	0.4903	
Implicit	F1	0.5816	

4.5 Classical CF versus deep learning

	Item-based CF	NCF (implicit)
NDCG@10	**0.1044**	0.0663
Training time	~3 s	464 s
Inference (800 users)	2.6 s	154 s
Parameters	—	278,817
Interpretable	yes — 'similar to X you bought'	no
Handles cold-start items	no	no
Scales to 1M users	no (O(users²) for user-CF)	yes (embeddings)

The honest finding

On this dataset, **classical item-based CF beats the deep model on accuracy at a fraction of the cost**. That is unsurprising at this scale: NCF's advantage lies in modelling complex interaction patterns that emerge with much larger and richer data, and 138k interactions across 6,000 customers is not enough signal to justify 278,817 parameters. The deep model still earns its place — it contributes 0.35 of the hybrid weight, and the hybrid beats item-based CF alone (0.1095 vs 0.1044). But a recommendation to deploy NCF *instead of* item-based CF would not be supported by this evidence.

4.6 Limitations

• **Synthetic data.** The models learned the rules the generator was written with. The metrics measure how well those specific rules were recovered. No claim is made that they transfer to real customer data.

• **Offline proxy.** Ranking metrics measure agreement with historical behaviour, which is itself a product of whatever ranked items in the past.

• **Cold start is unverifiable.** Customers with no history have no ground truth, so the fallback can be demonstrated but not scored.

• **Single split.** One time-based cutoff, not cross-validated. No confidence intervals are reported.

• **Calibration deviations.** Conversion runs at 16.4% against a real-world 2-3%, deliberately, to keep enough positive signal to train on.

Part 5 — Problems Faced

Every one of these was found by running and measuring the system, not by inspection. They are documented because the diagnosis is the most transferable part of the work.

Problem 1 — The deep model could not rank at all

Symptom. NCF scored **NDCG@10 = 0.0000** with catalogue coverage of 0.006. It returned the same ~14 items to every single customer.

Diagnosis. Not a tuning problem. The explicit model minimises error against *observed star ratings*. It had never once been shown an item a customer did **not** interact with, so scoring the unseen catalogue was completely outside its training distribution. It fell back on a global item bias — identical for everybody.

Fix. Train an implicit variant with negative sampling: 4 items the customer never touched, per observed positive. Now 'would this customer engage or not?' *is* the training objective. NDCG@10 went from 0.0000 to 0.0576.

> **Lesson**
> A model can be accurate on the metric you trained it on and useless at the task you actually need. RMSE 0.91 told us nothing about ranking quality.

Problem 2 — The hybrid was worse than its own components

Symptom. Hybrid NDCG@10 = 0.0760, but item-based CF alone scored 0.1044. The combined model was **losing to one of its own inputs**.

Diagnosis. The hybrid's collaborative slot was filled by SVD (0.0462), the *weakest* collaborative signal, while item-based CF — the strongest model in the whole system — was not in the fusion at all.

Fix. Re-ran the weight sweep and rebuilt the fusion around measured performance: item_cf 0.45 / svd 0.10 / content 0.10 / ncf 0.35. NDCG went from 0.0947 to 0.1293 on the sweep cohort.

> **Lesson**
> A hybrid is only as good as the signals you put in it. Fusing on intuition rather than measurement produced a model worse than its best part.

Problem 3 — The diversity re-ranker destroyed relevance

Symptom. After fixing a sign bug, the re-ranker pushed top-10 mean item exposure from 518 interactions down to **1**, with 100% long-tail share.

Diagnosis. Two separate bugs. First, **sign inversion**: SVD runs on the mean-centred matrix, so scores are negative for many items, and multiplying a negative by a penalty below 1 moves it *up* toward zero. Second, **wrong scope**: the penalty 1/(1+log(count)) spans a ~5x range, wider than the spread of the scores it multiplies, so it dominated the ranking.

Fix. Normalise to [0,1] before multiplying; switch to a gentler power-form penalty with a tunable alpha; apply it only to the top 200 candidates rather than the whole catalogue.

> **Lesson**
> A correction whose magnitude exceeds the signal it corrects is not a correction, it is a replacement.

Problem 4 — The API silently served without the deep model

Symptom. None. **That was the problem.**

Diagnosis. The API loaded NCF artifacts from app/saved_models/ — a directory that never existed — inside a bare try/except. It started, reported healthy, and served every recommendation with the deep-learning weight effectively zero. It also called .eval() on a state_dict, which would have failed even with the right path.

Fix. Load from the correct location; a missing model prints a loud warning naming the file and the fix command; /health reports ncf_available explicitly; fusion renormalises weights so a degraded hybrid stays internally consistent.

> **Lesson**
> A silent failure is worse than a crash. Anything optional must announce its absence.

Problem 5 — The Gini coefficient came out negative

Symptom. The long-tail validation check reported **Gini = −0.716**, which is mathematically impossible.

Diagnosis. The Gini integral is defined against the *ascending* Lorenz curve. The plotted concentration curve ('top X% of items drive Y% of demand') is sorted descending and sits above the diagonal. Integrating that one returns the value sign-flipped. The magnitude was correct, which is exactly why it was easy to miss.

Fix. Compute the Gini from a separately sorted ascending curve, leaving the plot as-is. Result: 0.716, and the check passes.

> **Lesson**
> A plausible-looking magnitude with the wrong sign is harder to spot than a wildly wrong number.

Problem 6 — The .gitignore had never worked

Symptom. 403 MB of model artifacts and compiled Python were tracked in git despite being listed in .gitignore.

Diagnosis. The file was saved as **UTF-16**. Git only parses UTF-8, so every rule in it had been silently inert since the repository was created.

Fix. Rewritten as UTF-8; artifacts untracked with git rm --cached. The repository went from a would-be 403 MB down to 10.9 MB.

> **Lesson**
> Encoding bugs in config files fail silently. The fix was one line; finding it required asking why a correct-looking file had no effect.

Problem 7 — The code could not run on this machine

Symptom. Every module failed on import.

Diagnosis. All paths were hardcoded to an absolute directory that did not exist on the current machine. A duplicate copy of every source file existed in reports/ purely to satisfy imports.

Fix. Every module now computes paths from its own file location using os.path. The duplicate files were deleted.

Lesson

Hardcoded absolute paths make a project unrunnable anywhere but the author's machine, and duplicated modules drift apart.

Part 6 — Viva Question Bank

Questions a reviewer is likely to ask, with answers grounded in the actual implementation and measured numbers.

6.1 On the problem and data

Q. Why synthetic data instead of MovieLens or a public dataset?

The business case specifies an early-stage platform where real interaction data is unavailable, and requires the system to be prototyped on synthetic data. Beyond compliance, generating the data means I control the ground truth — I know exactly which latent factors exist (category affinity and segment price sensitivity), so I can verify whether each model recovers them.

The trade-off is stated plainly: the models learned the rules the generator was written with, so the metrics do not transfer to real data. What transfers is the architecture and the evaluation methodology.

Q. How do you know your synthetic data is realistic?

Ten validation checks in notebooks/eda_validation.py, all passing. Sparsity 99.08%, top 10% of items holding 62.9% of demand, Gini 0.716, cold-start cohorts at 8% and 10.2%, and a J-shaped rating distribution with mean 3.91.

I also document two *deliberate* deviations. Conversion runs at 16.4% against a real-world 2-3%. At a realistic rate a 138k log yields only ~4,000 purchases, which is too sparse to train an implicit model or build a stable test set. That is a documented trade-off, not an oversight.

Q. Why is your matrix 99% sparse? Is that a problem?

It is realistic and intentional — real platforms are sparser still. It is the central difficulty of recommendation: two customers almost never rate the same item, so direct similarity has nothing to compare.

It is why the system uses latent-factor methods that generalise across related-but-not-identical items, and why content-based recommendation is included at all — it does not depend on the interaction matrix.

6.2 On the models

Q. Why did you use concatenation instead of element-wise product in NCF?

An element-wise product hard-codes a multiplicative interaction, which reduces the model back to matrix factorisation — you have added an MLP but constrained it to relearn a dot product.

Concatenation leaves the interaction function itself to be learned. That matters here because the data has a conjunctive rule: a customer buys when the category matches AND the price fits. A dot product cannot represent an AND.

Q. Your NCF scored NDCG@10 = 0.0000 at first. What happened?

The explicit model was trained to predict star ratings with MSE loss. It had never been shown an item a customer did not interact with, so ranking the unseen catalogue was out of distribution. It returned the global item bias — the same ~14 items to everyone, coverage 0.006.

The fix was negative sampling: draw 4 items the customer never touched per observed positive, and train with BCE. That makes discrimination across the catalogue the actual training objective. NDCG went from 0.0000 to 0.0576, accuracy from 0.534 to 0.858.

The broader lesson is that RMSE 0.91 looked like a healthy model. Rating accuracy and ranking quality are different things.

Q. Why does classical CF beat your deep learning model?

Because at this data scale it should. Item-based CF scores NDCG@10 = 0.1044 against NCF's 0.0663, trains in 3 seconds against 464, and is interpretable.

NCF's advantage is modelling complex interaction patterns that emerge with much larger, richer data. With 138k interactions across 6,000 customers there is not enough signal to justify 278,817 parameters.

I would not recommend deploying NCF instead of item-based CF on this evidence. But it earns its place inside the hybrid — it contributes 0.35 of the weight, and the hybrid beats item-CF alone at 0.1095 against 0.1044.

Q. How did you choose the hybrid weights?

By measurement. The first version used 0.35 / 0.25 / 0.40 across SVD, content and NCF and scored 0.0947 — worse than item-based CF alone at 0.1245. The hybrid was losing to its own strongest available input, which was not in the fusion at all.

I swept configurations over a 250-customer cohort and selected item_cf 0.45 / svd 0.10 / content 0.10 / ncf 0.35, which scored 0.1293. SVD stays at low weight because it generalises where co-rating neighbours are absent; content stays because its coverage is 75% against everything else's 20%.

6.3 On evaluation

Q. How do you guarantee no data leakage?

Three mechanisms. First, a time-based split — the final 90 days are held out and training uses only what came before. Second, every model is **refitted from scratch on the training period**; reusing the stored artifacts would have been faster but those were fitted on the full history. Third, assertions in the code that crash the benchmark if the temporal separation is ever broken.

The candidate set also excludes everything the customer touched during training, so no model scores points for 'predicting' something it was shown.

Q. Why NDCG rather than just precision?

Precision@10 treats a hit at position 1 and position 10 identically. On a storefront those are worth very different amounts because click-through decays sharply with position. NDCG applies a logarithmic positional discount, so it tracks revenue more closely.

I also use graded relevance, so a 5-star hit contributes more gain than a 4-star one.

Q. Why do you report coverage and diversity, not just accuracy?

Because a model can post excellent precision while only recommending 32 items out of 2,500. Commercially that is a failure — the rest of the inventory is invisible and will never sell. Accuracy metrics cannot detect it.

The popularity baseline has 1.3% coverage, and so did the broken NCF. Coverage is what exposed that failure.

6.4 On cold start and explainability

Q. How do you handle a brand-new customer?

Three tiers. A customer below the 3-interaction threshold is served from their registration profile — declared category, scored against their segment's price band. A completely unknown ID gets de-biased global popularity. A customer with history gets the full hybrid.

The price-band step matters: a Budget Shopper interested in Electronics is shown items around INR 2,000, not the most expensive product in the category, which is what a naive popularity fallback would do.

Every response carries a strategy field, so the caller always knows which path produced the result.

Q. How do you handle a brand-new product?

Content-based recommendation. A product listed this morning has a description, tags, category, brand and price, so it has a full TF-IDF vector before a single customer has clicked it. Collaborative filtering cannot score it at all by definition.

In the demo, cold-start item #3 with zero interactions resolves to Home & Kitchen neighbours at similarity 0.54, driven by shared category, brand and the terms 'dishwasher safe', 'storage'.

Q. Your similar-customer evidence is often empty. Is that a bug?

No — it is structural, and I surface it rather than hide it. The re-ranker deliberately promotes under-exposed long-tail items, and an item is long-tail precisely because few people have interacted with it. So the diversity mechanism erodes the collaborative evidence available to justify its own output.

When evidence is absent the system says so explicitly rather than producing a vague sentence. A customer told 'customers like you loved this' about an item nobody comparable has rated is being misled. When the item is *not* long-tail the evidence is strong — 19 of 21 neighbours rating it 4+ in the live demo.

6.5 On production and scale

Q. What breaks first as this scales?

User-user collaborative filtering. It is O(users²) — 133 MB at 5,766 users, but 40 GB at 100,000 and 4 TB at a million. It is quadratic in the fastest-growing dimension.

Item-item similarity is O(items²) and grows far more slowly because catalogues change more slowly than user bases. The migration path is to drop user-user CF entirely beyond ~50,000 users and move to an approximate nearest-neighbour index, then to two-stage retrieval where the full catalogue is never scored online.

Q. Batch or real-time?

Both, chosen per surface. Around 80% of traffic can be served from precomputed results — homepage rails, email, product-page 'similar items' — where hours-stale recommendations do no harm and serving is an O(1) lookup.

Real-time is reserved for surfaces where in-session behaviour genuinely changes the answer: post-add-to-cart suggestions and search re-ranking. Measured real-time latency is about 190 ms per request on CPU, dominated by the NCF forward pass.

Q. How would you monitor this in production?

Three layers. Model health: NDCG on a rolling holdout, catalogue coverage, cold-start fallback rate, and **per-user score variance** — that last one specifically because the NCF collapse showed up as near-identical scores for every customer, which is invisible to accuracy metrics.

System health: p50 and p99 latency, error rate, artifact load success. Business: CTR, conversion from recommendations, and share of revenue from long-tail items.

Plus fairness monitoring — coverage gap across activity bands, and whether the recommended price spread across segments is growing faster than the observed purchase spread.

Q. What would you do differently with real data?

Re-tune every hyperparameter. The current values are optimal for this generator, not for e-commerce in general.

Add exploration — epsilon-greedy or Thompson sampling — to break the popularity feedback loop, which offline metrics cannot capture. Run an online A/B test, which is the only way to validate cold-start quality since it has no offline ground truth. And build ingestion validation first: bot filtering in particular, because bot traffic directly poisons the popularity model and the similarity matrices.

Closing note

This project delivers a complete recommendation system for an e-commerce platform with no real interaction data — from synthetic data generation through to a served, explainable API.

The strongest evidence is not the headline metric. It is that the benchmark found three genuine modelling failures — a deep model that could not rank, a hybrid losing to its own input, and a diversity correction that inverted itself — and that each was diagnosed from measurement rather than inspection, then fixed and re-measured. A system that has been made to fail and then repaired is better understood than one that appeared to work first time.

Reproducibility

Every number in this document comes from a seeded pipeline. The full rebuild was verified from an empty data/processed/ directory and reproduced every metric exactly — RMSE 0.9108, accuracy 0.8583, 10/10 validation checks.